



M263 Course Handbook

Note: This handbook **may** be taken in to the M263 Examination.
Correction and annotation of this handbook **is** permitted.
Additional sheets may **not** be added to this handbook.

Contents

1	NOTATION	2
2	THE COURSE CODE	3
	Specification	4
	Code constructs	4
	Tracing code	6
	Notation for the WorkPad	7
3	BINARY TREES	8
4	LOGIC	10
	Boolean expressions	10
	Parse trees	11
	Logic and language	14
	Rules of Boolean algebra	15
	Rules of inference	15
	Proof by induction	16
5	DATABASES	17
	Functional dependency and keys	17
	Normal forms	19
6	RECURRENCE SYSTEMS	20
7	EFFICIENCY	21
8	AXIOMATIC SPECIFICATION	22
9	TYPE SPECIFICATIONS	23
	Primitive types	23
	Spot types	26
	Turtles	29
	Sequence based types	32
	Sets	36
	Binary trees	38
	Class implementations	39
10	GLOSSARY	42
11	DATA TYPE SUMMARIES	64
12	ASCII CODES FOR CHARACTERS	64

1 NOTATION

		Unit
[5, 3, 3, 17]	The sequence with members 5, 3, 3, 17 (in that order).	
	In general, square brackets delimit a sequence.	1
[]	The empty sequence.	1
'&'	The character '&'. Single inverted commas denote a character.	1
"abc"	The string "abc", which is equivalent to the sequence of characters ['a', 'b', 'c']. Double inverted commas denote a string.	1
""	The empty string.	1
'□'	The space character.	1
(x, y)	An ordered pair. Round brackets delimit a tuple.	1
(x, y, z)	An ordered triple, or 3-tuple.	1
$X \times Y$	The Cartesian product of the sets X and Y (" X cross Y ").	1
$X \amalg Y$	The disjoint union of the sets X and Y (" X or Y ").	1
Int	The set of all whole numbers; positive, negative and zero.	1
Char	The set of characters with ASCII codes between 32 and 126.	1
Bool	The set {true, false} .	1
$\text{SeqOf}X$	The set of all sequences whose members come from X .	1
$\text{SetOf}X$	The set of all sets whose members come from X .	1
SeqOfSetOfInt	The set consisting of all sequences whose members are sets of integers, e.g. [{1}, {2, 4}, {8, 16, 32}].	1
$\text{DIV}(m, n)$	The result of integer division of m by n . (e.g. $\text{DIV}(13, 4) = 3$).	1
$\text{MOD}(m, n)$	The remainder on integer division of m by n (e.g. $\text{MOD}(13, 4) = 1$).	1
$n!$	("n factorial") The product of the integers from 1 to n .	13
$a \wedge b$	("a and b") The binary operation "and" on Boolean values.	1
$a \vee b$	("a or b") The binary operation "or" on Boolean values.	1
$m \leq_{\text{Int}} n$	("m is less than or equal to n") Subscripts on operations may be used to emphasise the form of data being compared, but are not required.	2
$c \geq_{\text{Char}} d$	("c is greater than or equal to d") Comparison of two characters by reference to their ASCII codes.	2
{1, 3, 7}	The set whose members are the integers 1, 3 and 7.	
	Curly brackets are used to delimit the members of a set.	7
{ }	The empty set.	7
$\{x \text{ in } X : p(x)\}$	The set of all members of X for which $p(x)$ is true .	7
$x \in A$	The proposition: x is a member of the set A .	7
$A \subseteq B$	The proposition: A is a subset of B (every element in A is also in B).	7
$A \subset B$	The proposition: A is a proper subset of B ($A \subseteq B \wedge \neg(A == B)$).	7
$A \cup B$	The union of A and B ; the set containing all items in A or B .	7
$A \cap B$	The intersection of A and B ; the set of items in both A and B .	7
$A - B$	The set difference of A and B ; the set of items in A and not in B .	7
$O(f(n))$	A function is $O(f(n))$ if it is asymptotically dominated by $f(n)$.	15
$\Theta(f(n))$	A function is $\Theta(f(n))$ if it is of order $f(n)$.	15
$\neg p$	Negation of a proposition p , read as "not p ".	9
$p \wedge q$	The proposition " p and q ".	9
$p \vee q$	The proposition " p or q ".	9
$p \Rightarrow q$	The proposition " p implies q ".	9
$p \Leftrightarrow q$	The proposition " p if and only if q ".	9
$\exists$	The existential quantifier, read as "there exists".	10
$\forall$	The universal quantifier, read as "for all".	10
$\rightsquigarrow$	$A \rightsquigarrow B$ may be read as "A determines B", and means that for each attribute x in the set B , x is functionally dependent on A .	11
$\frac{s_1; s_2}{t}$	A deduction: if both s_1 and s_2 are true , then we can deduce that t is true .	10

2 THE COURSE CODE

Notation associated with the course code

		Unit
$\leftarrow$	Used for assignment (replaced by <code><--</code> in the WorkPad).	2
var x in Int	A declaration of the type of the variable x (that x is of type Int).	2
if	Precedes a condition.	2
then	The code following then is executed when the if condition is true .	2
else	The code following else is executed when the if condition is false .	2
for ($i \leftarrow n$ to m)	The following code is to be executed repeatedly, with i equal to n , then $n + 1$, then $n + 2$, ... up to i equal to m .	2
while ($condition$)	The following code is repeated so long as $condition$ evaluates to true .	6
true	A literal of type Bool .	3
false	A literal of type Bool .	3
'a'	A literal of type Char .	3
117	A literal of type Int (the integer is expressed in decimal notation).	3
[22, -31, 15, 15, 0]	A literal of type Array of Int .	3
$\perp$	Represents a blank in an array when tracing an array variable.	3
var arr in Array (of size n) of X	Used to declare an array variable arr with n places in the array, which contains items of the data type X .	3

Curly brackets { and } are used to delimit the statements forming the body of a loop, or the code to be executed following **then** or **else**.

The condition following **if** or **while** must be enclosed in round brackets.

An implementation of a function (say *AFUNCTION*), with inputs *inOne* and *inTwo* (say) must start with a statement

function *AFUNCTION*(*inOne*, *inTwo*)

and the code statements forming the remainder of the implementation are delimited by curly brackets { and }.

Literals and setter methods

The notation for literals of types **Bool**, **Char**, **Int** and **Array** are shown above. Note than an array literal can *not* contain blanks.

Stack of X has no literals, but you can use *SETSTACK* (e.g. $s \leftarrow SETSTACK(1, 2, 3, 4, 5)$ sets the state of s to [1, 2, 3, 4, 5]).

Class types have no literals, but some have setter methods:

- for **Vector**, $v.setVector(1, 2, 3)$ sets the state of v to [1, 2, 3]
- for **mString**, $s.setString("Qwerty")$ sets the state of s to "Qwerty"
- for **Set**, $a.setSet(1, 2, 3, 2)$ sets the state of a to {1, 2, 3}.

In the WorkPad, $t \leftarrow getTree()$ sets t to a tree entered by the user, using **Binary tree|Draw**.

Spot and **Turtle** have neither literals nor setter methods.

Specification

In M263, functions must not have side effects: any function returns a value, and calling the function must not change the state of the variables input to the function. A function specification is laid out as in the example below. The first line is the signature line, the second is the precondition, and the third is the postcondition.

```
function AT(i in Int, arr in Array of X) return in X  
pre    The array arr is full and  $1 \leq i$  and  $i \leq \text{SIZE}(arr)$ .  
post   The returned value is the ith item in the sequence arr.
```

Where a function is a binary operation using infix notation, then the signature line in the specification is given in a slightly different form.

```
function (infix) (c > d on Char) return in Bool  
pre    true.  
post   The returned value is true if  $ASC(c) > ASC(d)$  and is false otherwise.
```

A method is part of a class and the type of receiver object must be that class. So a method specification does not need to give the type of the receiver object. It must state whether the method changes the state of the receiver object, or returns a value. Examples (taken from the class **QuickSpot**) are given below. (In M263, all methods *either* return a value *or* change the state of the receiver — they do not do both.)

```
method getYPos() return in Int  
pre    true.  
post   For a spot with state (X, Y), the returned value is Y.  
  
method upQuick(n in Int) state change  
pre     $n > 0$ .  
post   The quickspot is moved up by n positions.
```

Apart from Unit 12.

Code constructs

Variable declarations Examples of variable declarations are given below. The first declares a variable *aVar* and states the type of *aVar* to be **ThisType**; the second declares an array variable *arr* of size *n* and containing items of type **X**.

```
var aVar in ThisType  
var arr in Array (of size n) of X
```

Default values If the data type **ThisType** has a default value, then this gives the state of the variable *aVar* after a **var** statement. If **ThisType** has no default value (as is the case for **Int**, **Char** and **Bool**), then the state of *aVar* is indeterminate until it is initialised.

Assignment	
Course code	Meaning
$a \leftarrow b$	For primitive types this changes the state of a to become the current state of b . The state of b is unchanged. Later changes to the state of b do not affect the state of a (and vice versa).
$a \leftarrow b.clone()$	For class types this changes the state of a to become the current state of b . The state of b is unchanged. Later changes to the state of b do not affect the state of a (and vice versa).

Conditionals	
Course code	Meaning
if ($condition$) then { $code$ }	This performs $code$ if $condition$ is true . If $condition$ is false , then $code$ is not executed.
if ($condition$) then { $codeOne$ } else { $codeTwo$ }	This performs $codeOne$ if $condition$ is true , and performs $codeTwo$ if $condition$ is false .

Loops	
Course code	Meaning
for ($index \leftarrow m$ to n) { $code$ }	$code$ (the body of the for loop) is repeated once for each value of $index$ from m to n , inclusive.
while ($condition$) { $code$ }	$code$ (the body of the while loop) is repeated so long as $condition$ evaluates to true .

The statements in the body of a **for** loop are repeated with $index$ equal to, in turn, m , then $m + 1$, then $m + 2$, and so on, until $index$ is equal to n . (If $m = n$, then the body of the loop is executed just once, with $index = m = n$.) If $m > n$, then the statements in the body of the loop are never executed. The variable $index$ is referred to as the **loop variable**. The loop variable should be declared before the loop, but should not be initialised, or used outside the body of the loop.

Before the body of a **while** loop is executed, $condition$ is evaluated using the current states of the variables. If $condition$ evaluates to **true**, then the body of the **while** loop is executed (after which $condition$ is evaluated again). If $condition$ evaluates to **false**, then execution of the **while** loop stops.

Validity

Examples of invalid course code include:

- using a variable that has not been created through a **var** statement;
- attempting to store data of the wrong type in a variable;
- calling a function in a way that is not consistent either with its signature or its precondition.

Tracing code

Notation for showing states

Type	Example	
Spot	$(3, -4)$	In traces, we use the usual notation for a sequence for stacks, arrays and vectors; blanks in an array can be shown as $\perp$.
ColourSpot	$(3, -4, \text{Red})$	
Marker	$(-2, 1, \text{"up(90)"})$	
Stack of X	$[1, 2, 3]$	In sets, it is best to avoid repeating elements.
Array of X	$[5, \perp, 8, \perp]$	
Vector of X	$[13, 2, 0, -1]$	The state of a turtle or a binary tree is usually shown as a diagram.
mString	"abc"	
Set of X	$\{3, 5, 2, 9\}$	

Examples

Code	<i>s</i>
var <i>s</i> in Stack of Int	$[\]$
<i>s</i> $\leftarrow$ <i>SETSTACK</i> (1, 2, 3)	$[1, 2, 3]$
<i>s</i> $\leftarrow$ <i>POP</i> (<i>s</i>)	$[1, 2]$
<i>s</i> $\leftarrow$ <i>PUSH</i> (5, <i>s</i>)	$[1, 2, 5]$

For simple code, we can trace the states of the variables after each statement.

Loops For code with a loop, a trace usually just gives the states after each execution of the loop body. The code to the right has an input *v* that is a vector; the trace below is when *v* is $[11, 22, 33, 44]$.

Comment	<i>index</i>	<i>v.at(i)</i>	<i>sum</i>
<i>sum</i> $\leftarrow$ 0	—	—	0
loop	1	11	11
	2	22	33
	3	33	66
loop end	4	44	110

```

function ADDALL(v)
{
  var sum in Int
  var i in Int
  sum  $\leftarrow$  0
  for (i  $\leftarrow$  1 to v.size())
  {
    sum  $\leftarrow$  sum + v.at(i)
  }
  return sum
}

```

Recursive code For recursive code, a trace usually just gives a value (with pending calls) after each recursive call. The code to the right has an input *v* that is a vector; the trace below is when *v* is $[11, 22, 33, 44]$.

```

ADDALL( $[11, 22, 33, 44]$ )
= 44 + ADDALL( $[11, 22, 33]$ )    (by R)
= 44 + (33 + ADDALL( $[11, 22]$ )) (by R)
= 44 + (33 + (22 + ADDALL( $[11]$ ))) (by R)
= 44 + (33 + (22 + (11 + ADDALL( $[\ ]$ )))) (by R)
= 44 + (33 + (22 + (11 + 0)))    (by S)
= 44 + (33 + (22 + 11))
= 44 + (33 + 33) = 44 + 66
= 110.

```

```

function ADDALL(v)
{
  var temp in Vector of Int
  var n in Int
  temp  $\leftarrow$  v.clone()
  if (temp.size() == 0) then
  { n  $\leftarrow$  0 } //(S)
  else
  {
    n  $\leftarrow$  temp.last()
    temp.removeLast()
    n  $\leftarrow$  n + ADDALL(temp)    //(R)
  }
  return n
}

```

In Unit 15, fuller traces are used — to show every statement execution.

Notation for the WorkPad

You can edit code for the WorkPad in a plain text editor. You can also use Microsoft Word, for example, and then copy it into the WorkPad. If you do this, you need to remember the following.

Only symbols available on the keyboard are used. In particular, we use `<--` for assignment. The WorkPad will ignore formatting such as bold or italic. Turn off “smart quotes”, or the WorkPad will not recognise quotation marks.

Certain binary operations are denoted in ordinary text by special symbols not available on the keyboard. These are entered into the WorkPad using one or more keyboard symbols, as shown below.

Operation	Symbol in text	In WorkPad
multiplication in Int	$\times$	<code>*</code>
greater than or equal to	$\geq$	<code>>=</code>
less than or equal to	$\leq$	<code><=</code>
“and” on Bool	$\wedge$	<code>&\</code>
“or” on Bool	$\vee$	<code>\ </code>

Reserved words and data type names should not be used as identifiers for variables used in code. Nor should literals such as **true** and **false**.

The following are reserved words in the course code.

var in on if then else return
function for to of extends class calls
this method while

The following are data type names.

Bool Array Spot Turtle Vector Set
Int Stack QuickSpot TurtlePlus VectorPlus BinTree
Char ColourSpot SuperTurtle mString

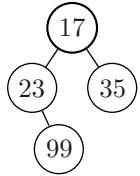
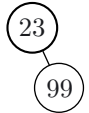
Comments in code may be delimited by starting with `/*` and ending with `*/`, or a comment can be preceded by `//` if it is on a single line. Comments are used to explain what code is doing. They are not addressed to the computer, which cannot interpret them and will just ignore them, but rather to a person trying to follow the code.

When an invalid expression is entered into the WorkPad, a variety of things may happen.

- The code may fail completely.
- The code may fail, and some error message be returned (which may — or may not — be informative).
- The expression may return some value.

If the last happens, the value returned will be unpredictable. For invalid code, there is no “correct” value that can be returned, so you have no way of telling what value might be returned by invalid code.

3 BINARY TREES

In the binary tree $t =$ , the left subtree is , the right

subtree is , and the root is 17.

In the **linear notation** for a binary tree, the root is written first, then the left subtree, then the right subtree. The linear notation for  is:
 $(\ 35\ (\)\ (\)\)$.

The linear notation for t is:

$(\ 17\ (\ 23\ (\)\ (\ 99\ (\)\ (\)\)\)\ (\ 35\ (\)\ (\)\)\)\)$.

To form a complete tree from a sequence s

1. Work from the left of the sequence s .
2. Put the first item at the root node of the tree.
3. Add the second item as the left child of the root and the third item as the right child of the root.
4. Move to level 3 in the tree, and fill that starting at the left.
5. Continue until all the items in the sequence are in the tree.

To form
 $COMPLETE_TREE(s)$.

Level order tree traversal

Given a tree, we form a sequence containing the items at its nodes as follows. (The sequence is written down going from left to right.)

First, the root.

Then the nodes at level 2, working from left to right.

Then the nodes at level 3, working from left to right.

This process is continued, going down one level at a time, until all the nodes in the tree have been visited.

To form $LEVEL_ORDER(t)$.

In-order tree traversal

Draw a line around the outside of the tree. Start to the left of the root, and go around the outside of the tree, to end up to the right of the root. Stay as close to the tree as possible, but do not cross the tree. (Think of the tree — its branches and nodes — as a solid barrier.) The order of the nodes is the order in which this line passes underneath them. If you are unsure as to when you go “underneath” a node, remember that a node “to the left” always comes first.

To form $IN_ORDER(t)$.

Alternatively, “pancake” the tree.

We can recognise a **binary search tree** (BST) as follows.

t is a BST if and only if $IN_ORDER(t)$ is a sorted sequence.

Also, $GROW_TREE(v)$ is a BST if and only if v is a sorted sequence.

Growing a tree from a sequence

1. If the sequence is empty then the empty tree is returned.
2. If the sequence is not empty, first find the centre of the sequence. (If the length of the sequence is even, then take the right-hand of the two candidates.)
3. Set this central element at the root of the returned tree.
4. The central element divides the sequence into a left sub-sequence and a right sub-sequence (one or both of which may be empty).
5. Apply the same procedure to the left sub-sequence to produce the left subtree of the returned tree.
6. Apply the same procedure to the right sub-sequence to produce the right subtree of the returned tree.

To form $GROW_TREE(v)$.

function $SEARCH_TREE(v$ in Vector of X) **return** in BinTree of X
pre true.

post The returned value is the tree obtained by first sorting v , then applying $GROW_TREE$ to the resulting vector.

Adding an element to a BST

To add $elem$ to a BST t , proceed as follows. Move down through the tree t until we reach an empty subtree, moving as follows.

1. Compare $elem$ to the root of t .
If $elem \geq$ the root of t , move down and right.
Otherwise, move down and left.
2. Compare $elem$ with the current node of t , say $node$.
If $elem \geq node$, move down and right.
Otherwise, move down and left.
3. Continue moving down until you come to an empty subtree.
This is the place to add $elem$ to t .

Adding an item to a heap

To add a new item x to a heap tree, proceed as follows.

1. Add x to the heap at the first available place that keeps a complete tree (x will now be at a leaf node).
2. If the new item x is less than the item at its parent node then exchange these items.
3. Repeat the process of comparing the new item with the current parent node and exchanging if necessary.
4. Stop when the new item is greater than (or equal to) its parent.

Removing the root item from a heap

1. Delete the old root.
2. Place the value of the “last leaf” node at the root. This is referred to as the “sinking item”.
3. Remove the last leaf position from the tree.
4. Starting with the sinking item at the root, repeat the following:
 - 4.1 find the smallest child of the current position of the sinking item;
 - 4.2 if this smallest child is smaller than the sinking item, then exchange it with the sinking item.
5. The repetition stops when the sinking item is smaller than (or equal to) all its children (or it has no children).

Comparing strings

1. Compare the strings s and t character-by-character, starting at the front. Use the first characters where the strings are different to determine whether or not $s < t$ is **true**.
2. If s is identical to the front of t , and s is shorter than t , then $s < t$ is **true**.
3. If t is identical to the front of s , and t is shorter than s , or if t is identical to s , then $s < t$ is **false**.

To form $LESS_THAN(s, t)$.

4 LOGIC

Sets

Let A and B be two sets from the predefined set U . Then

$$A \cup B = \{x \text{ in } U : (x \in A) \vee (x \in B)\}$$

$$A \cap B = \{x \text{ in } U : (x \in A) \wedge (x \in B)\}$$

$$A - B = \{x \text{ in } U : (x \in A) \wedge \neg(x \in B)\}$$

Let $p(x)$ and $q(x)$ be predicates about the set U . Then

$$\{x \text{ in } U : p(x)\} \cup \{x \text{ in } U : q(x)\} = \{x \text{ in } U : p(x) \vee q(x)\}$$

$$\{x \text{ in } U : p(x)\} \cap \{x \text{ in } U : q(x)\} = \{x \text{ in } U : p(x) \wedge q(x)\}$$

$$\{x \text{ in } U : p(x)\} - \{x \text{ in } U : q(x)\} = \{x \text{ in } U : p(x) \wedge \neg q(x)\}$$

Boolean expressions

The truth table for the Boolean expression $a \vee (b \wedge \neg a)$ is given below.

a	b	a	$\vee$	$(b$	$\wedge$	$\neg a)$
T	T	T	T	T	F	F
T	F	T	T	F	F	F
F	T	F	T	T	T	T
F	F	F	F	F	F	T

which gives

a	b	$a \vee (b \wedge \neg a)$
T	T	T
T	F	T
F	T	T
F	F	F

disjunctive normal form is
 $(a \wedge b) \vee (a \wedge \neg b) \vee (\neg a \wedge b)$

To draw up the truth table for a Boolean expression

1. Determine the number of atoms in the expression and draw up a truth table with the left-hand columns headed by these atoms.
2. Fill these columns with T and F arranged systematically.
3. Write the full expression to the right of the variable headings with each term and each operation as a column heading. (Treat an expression of the form $\neg a$ as a single term.)
4. Use the brackets to determine the order of precedence of the operations and number them accordingly.
5. Calculate the values in each of these columns in turn.
6. When your calculation is finished, you may like to redraw the table showing the left-hand columns and the column giving the truth values of the expression.

T for true; F for false.

Boolean expressions with the property that the truth values of the two expressions are the same for every combination of the truth values of the atoms in the expressions are said to be **equivalent**. We write $r \equiv s$ when the expressions r and s are equivalent.

The **disjunctive normal form** of a Boolean expression contains one term for each row of its truth table that evaluates to **true**. It is always equivalent to the original expression. It is formed as follows.

- Find its truth table.
- Write down the term corresponding to each row evaluating to **true**.
- Connect these terms by $\vee$.

Parse trees

From parse tree to expression

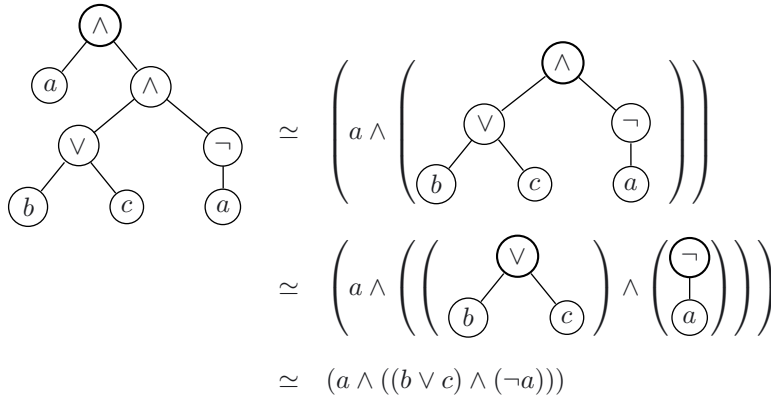
In a parse tree, each leaf node is an atom. Each non-leaf node is an operation, and the branches from such nodes lead either to an atom or to a subtree that is again a valid parse tree. If the operation is $\vee$ or $\wedge$, the node has two branches, while if it is $\neg$, the node has one branch.

A parse tree is not a binary tree.

We can find the expression represented by a parse tree working “bottom up” as follows.

- Step 1:** Start at the lowest level of the tree, and look at the nodes whose branches lead to leaf nodes. Replace each node at this level by the expression that it represents. Where an expression has been substituted for a subtree, place the expression in brackets, so that it is treated as a single item at the next stage.
- Step 2:** Proceed up the levels of the tree, replacing operations by expressions as required.
- Step 3:** The final step is to replace the root node. The parse tree represents the expression now found in the root node.

Using the “top down” approach we start from the root node, as below.



The notation $\simeq$ here can be read: “can be written as”. The large brackets around the trees represent brackets in the final expression being found. As each node is dealt with, brackets are introduced as necessary.

We can use a parse tree to **find the truth value** of the expression it represents, for given truth values of the atoms in the expression. To do this, put the truth values for the atoms on the branches leading from them, and then work up the tree finding the value at each node in turn.

Drawing a parse tree for an expression

Find the root node (see below). From this you can identify either one or two subexpressions, i.e. parts of the original expression that can be represented as parse trees. Find the root nodes of these. Continue until the subexpressions found are atoms.

Finding the root node for an expression

- Insert brackets around expressions of the form $\neg r$, wherever they have been omitted. (Each $\neg$ should have an opening bracket directly before it in the expression.)
- Remove any outer brackets.
- Start with count 0. Work through the expression from left to right. Add 1 to your count for each opening bracket, and subtract 1 for each closing bracket.
- Note the count when you pass an operation symbol ($\wedge$, $\vee$ or $\neg$).
- The root node will be the operation where the bracket count is zero.

Parse tables

We call a Boolean expression a **well-formed formula** if it is built up according to the rules given below. Any formula built up using these rules is certain to be syntactically correct.

atom	Atoms, such as $a, b, c, \dots$ are well-formed formulae.
negation	If s is a well-formed formula then $(\neg s)$ is well-formed.
conjunction	If s, t are well-formed formulae then $(s \wedge t)$ is well-formed.
disjunction	If s, t are well-formed formulae then $(s \vee t)$ is well-formed.
implication	If s, t are well-formed formulae then $(s \Rightarrow t)$ is well-formed.
equivalence	If s, t are well-formed formulae then $(s \Leftrightarrow t)$ is well-formed.

A **parse table** presents in tabular form the process of building up a well-formed formula — an example is given below. In the table, we give a reference number for each well-formed formula that we form along the way. These numbers are in the left-most column in the table.

Number	Formula	How formed
1	b	atom
2	$(\neg b)$	negation(1)
3	c	atom
4	$((\neg b) \vee c)$	disjunction(2, 3)
5	a	atom
6	$(a \wedge ((\neg b) \vee c))$	conjunction(5, 4)

Conditional statements and outcomes

An **outcome table** is a table that shows the outcomes for a conditional statement for all possible combinations of values of the Boolean variables. The outcome table gives rise to an **outcome tree**. Where adjacent branches in an outcome tree lead to the same outcome, the tree can be simplified.

Constructing conditional statements

1. Define propositions that control the desired outcome.
2. Draw up an outcome table.
3. Draw and simplify the outcome tree.
4. Use the outcome tree to construct the conditional statement.

Logic and language

Truth tables for connectives: $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$ and $\neg$

a	b	$a \wedge b$	a	b	$a \vee b$	a	b	$a \Rightarrow b$	a	b	$a \Leftrightarrow b$	a	$\neg a$
T	T	T	T	T	T	T	T	T	T	T	T	T	F
T	F	F	T	F	T	T	F	F	T	F	F	F	T
F	T	F	F	T	T	F	T	T	F	T	F	F	T
F	F	F	F	F	F	F	F	T	F	F	T	F	T

Propositions are statements expressed in English which may be **true** or **false**. They can be combined using the Boolean operations. The following are direct translations of such **composite propositions**:

Symbols	Translation	Truth values
$\neg p$	Not p	true if p is false
$p \vee q$	Either p or q (or both)	true if either or both of p or q is true
$p \wedge q$	p and q	true if both p and q are true
$p \Rightarrow q$	If p then q	true if we never have p true and q false
$p \Leftrightarrow q$	p if and only if q	true if p and q have the same truth values

Predicates and variables

The term **predicate** is used to refer to a property of a single item or to a relationship between items. For example, for a string s and character c , suppose that $FIRST(s, c)$ means that c is the first character in s . Then $FIRST$ is a predicate. Formally, $FIRST(s, c)$ is a **predicate sentence**, but we usually just refer to $FIRST(s, c)$ as a predicate.

Quantifiers

The **universal quantifier**, $\forall$, can be read as “for all”.

The **existential quantifier**, $\exists$, can be read as “there exists” (or “there is”, or “for some” or “there is some”).

The variable following either quantifier must come from a set within the domain of interest. The notation we use shows which set in the domain of interest the variable must come from.

The **binding term** consists of the quantifier, variable and set to which the variable belongs. The following part of the expression, in square brackets, is known as the **scope** of the quantifier. The square brackets show the extent of the relationship between the quantifier and the variable that it binds. The variable is bound by the quantifier within the scope of the quantifier.

For example, suppose that $Dict$ is a given set of strings and $FIRST$ is as above. Then the proposition

$\forall s \text{ in } Dict[FIRST(s, 'q')]$

has the interpretation “Every string in the set $Dict$ begins with the character ‘q’.”. In this proposition, $\forall s \text{ in } Dict$ is the binding term, and $[FIRST(s, 'q')]$ is the scope.

A proposition $\exists x \text{ in } U[p(x)]$ is **true** if there is a value x in the set U for which the proposition $p(x)$ is **true**.

A proposition $\forall x \text{ in } U[p(x)]$ is **true** if the proposition $p(x)$ is **true** for every value x in the set U .

Rules of Boolean algebra

Commutative laws $s \vee t \equiv t \vee s$ $s \wedge t \equiv t \wedge s$ Associative laws $s \vee (t \vee r) \equiv (s \vee t) \vee r$ $s \wedge (t \wedge r) \equiv (s \wedge t) \wedge r$ Distributive laws $s \vee (t \wedge r) \equiv (s \vee t) \wedge (s \vee r)$ $s \wedge (t \vee r) \equiv (s \wedge t) \vee (s \wedge r)$	Idempotent laws $s \vee s \equiv s$ $s \wedge s \equiv s$ Absorption laws $s \vee T \equiv T$ $s \vee F \equiv s$ $s \wedge T \equiv s$ $s \wedge F \equiv F$	Rules about negation $s \vee \neg s \equiv T$ (i) $s \wedge \neg s \equiv F$ (ii) $\neg \neg s \equiv s$ (iii) De Morgan's laws $\neg(s \vee t) \equiv \neg s \wedge \neg t$ (i) $\neg(s \wedge t) \equiv \neg s \vee \neg t$ (ii) Rewriting $\Rightarrow$ $s \Rightarrow t \equiv \neg s \vee t$ $s \Leftrightarrow t \equiv (s \Rightarrow t) \wedge (t \Rightarrow s)$
--	--	---

In each of the rules, s , t and r can be single atoms, or they can be Boolean expressions.

Extended commutativity

$$\forall x \text{ in } U[\forall y \text{ in } V[p(x, y)]] \equiv \forall y \text{ in } V[\forall x \text{ in } U[p(x, y)]]$$

$$\exists x \text{ in } U[\exists y \text{ in } V[p(x, y)]] \equiv \exists y \text{ in } V[\exists x \text{ in } U[p(x, y)]]$$

Extended De Morgan

$$\neg \forall x \text{ in } U[p(x)] \equiv \exists x \text{ in } U[\neg p(x)] \quad (\text{i})$$

$$\neg \exists x \text{ in } U[p(x)] \equiv \forall x \text{ in } U[\neg p(x)] \quad (\text{ii})$$

Rules of inference

Implication	$\frac{s; s \Rightarrow t}{t}$	Disjunction	$\frac{\neg s; s \vee t}{t}$	Addition	$\frac{s}{s \vee t}$
Contraposition	$\frac{\neg t; s \Rightarrow t}{\neg s}$	Combining $\vee, \Rightarrow$	$\frac{s \vee t; s \Rightarrow r}{r \vee t}$	Conjunction	$\frac{s; t}{s \wedge t}$
Equivalence	$\frac{s \Leftrightarrow t; s}{t}$	Transitivity	$\frac{s \Rightarrow t; t \Rightarrow r}{s \Rightarrow r}$	Simplification	$\frac{s \wedge t}{s}$
and	$\frac{s \Leftrightarrow t; t}{s}$				

In formal logic, an **argument** is an assertion that a set of propositions $s_1, s_2, \dots, s_n$ yields another proposition t . Such an argument is a **valid argument** if $(s_1 \wedge s_2 \wedge \dots \wedge s_n) \Rightarrow t$ is a tautology.

In an argument corresponding to an expression of the form

$$(s_1 \wedge s_2 \wedge \dots \wedge s_n) \Rightarrow t,$$

we call $s_1, s_2, \dots, s_n$ the **premises** of the argument and t the **conclusion** of the argument. If the expression is a tautology, so that we have a valid argument, then t is called a **semantic consequence** of

$$s_1 \wedge s_2 \wedge \dots \wedge s_n, \text{ and we write } \frac{s_1; s_2; \dots; s_n}{t}.$$

To show an argument is valid, list propositions that are **true**, ending with the desired conclusion. For each proposition that you write down, justify its inclusion either by indicating that it is a premise or by giving the rule of inference that allows its truth to be concluded.

A set of propositions $p_1, p_2, \dots, p_n$ is called **consistent** if it is possible for $p_1 \wedge p_2 \wedge \dots \wedge p_n$ to be **true**.

A Boolean expression is a **contradiction** if it is equivalent to **false**. So the set of propositions $p_1, p_2, \dots, p_n$ is not consistent if $p_1 \wedge p_2 \wedge \dots \wedge p_n$ is a contradiction.

Proof by induction

Principle of mathematical induction (Version 1)

Suppose that $p(n)$ is a proposition depending on n , where n is an integer and $n \geq 1$.

Suppose that we can prove two things, namely:

- (B) $p(1)$ is **true**;
- (I) for all integers $k > 1$, $p(k-1) \Rightarrow p(k)$ is **true**.

Then we can deduce that $p(n)$ is **true** for all integers $n \geq 1$.

(B) may be called the **starting step**, or the **starting condition**, or the **basis for induction**, and (I) is the **inductive step**. The principle can be extended in various ways — in particular, to start at a general integer M rather than 1.

The inductive step consists of an implication as shown. To prove this implication, we suppose that $p(k-1)$ (the left-hand side of the implication) holds and prove that the right-hand side must follow. In establishing the implication in (I), the assumption that $p(k-1)$ holds is known as the **induction hypothesis**, from which the truth of $p(k)$ must be deduced. Once we have established both (B) and (I), we can use mathematical induction to deduce that $p(n)$ holds for all $n \geq 1$.

To prove an implementation using a while loop is correct

1. Check that the states of the variables just before the loop is entered satisfy the proposed loop invariant condition.
2. Check that a single execution of the loop body does not change the value of the proposed loop invariant.
3. The loop ceases execution when the **while** condition becomes **false**. Use this fact, and the proven loop invariant condition, to discover information about the states of the variables just after the loop finishes execution.

A **loop invariant condition** is a condition that holds after each execution of the loop body is completed. To use induction to establish that a loop invariant condition holds, show that:
it holds just before the loop is entered; and,
if it holds after $k-1$ executions (where $k > 0$), then it also holds after k executions.

To do this, we often use the following approach. Suppose that the local variables are $vOne$ and $vTwo$. Denote the values of these variables after k executions of the loop body by $vOne_k$ and $vTwo_k$. Use the code in the loop body to express $vOne_k$ and $vTwo_k$ in terms of $vOne_{k-1}$ and $vTwo_{k-1}$, and then use these relationships to show that the proposed loop invariant condition must hold.

5 DATABASES

In a table, the column headings are called **attributes**, and show what feature of a data item is stored in that column. A row is called a **tuple**. Each tuple gives a data item — an entry in the database. A collection of tuples forming a complete table is a **relation**. Each attribute has a **domain** and a **type**.

We use the term **relation scheme** to mean a set of attributes *together with* any underlying rules that apply (i.e. the functional dependencies).

Functional dependency and keys

Let A be a set of attributes and x be another attribute of the scheme. Suppose that the following holds.

In a relation over the scheme, if two tuples have the same value for all the attributes in A , then they must also have the same value of x .

We then say that x is **functionally dependent** on the set of attributes A , and write $A \rightsquigarrow \{x\}$.

Properties of functional dependencies — Result 11.2.1

If X, A, B and C are sets of attributes of the same scheme, then

1. $X \rightsquigarrow X$.
2. If $X \rightsquigarrow A$ and $X \rightsquigarrow B$ then $X \rightsquigarrow A \cup B$.
3. If $X \rightsquigarrow A$ and $C \subseteq A$ then $X \rightsquigarrow C$.
4. If $X \rightsquigarrow A$ and $X \subseteq C$ then $C \rightsquigarrow A$.
5. If $A \rightsquigarrow B$ and $B \rightsquigarrow C$ then $A \rightsquigarrow C$. (**Transitivity**.)

We can separate the functional dependencies of a relation scheme into two groups. Those that follow directly from the rules of the scheme are the **explicit functional dependencies**. The remaining functional dependencies, which can be obtained using the properties above, are **implicit functional dependencies**. These implicit functional dependencies include **trivial** dependencies such as $\{x\} \rightsquigarrow \{x\}$ (which would hold in any scheme with an attribute x), and any dependencies that can be deduced from the explicit and trivial dependencies using rules like transitivity.

In a relation scheme, a subset K of the set of attributes Att is a **key** for the relation scheme if both of the following properties hold.

1. For every attribute x in Att , $K \rightsquigarrow \{x\}$. (Determining)
2. There is no proper subset $Y \subset K$ with the property that $Y \rightsquigarrow \{x\}$ for all x in Att . (Minimal)

A relation scheme may have several keys, in which case it is usual to choose one as a **primary key**.

Finding implicit dependencies that come from transitivity

1. Augment each explicit functional dependency by adding all the attributes from the left-hand side to the right-hand side.
2. For each augmented dependency, check whether the left-hand side is a subset of the right-hand side of any other dependency.
3. If it is, then you will have dependencies $X \rightsquigarrow Y$ and $W \rightsquigarrow Z$ where $W \subseteq Y$. Add $X \rightsquigarrow Z$ to your list. Then augment this new dependency, as in step 1.
4. Amalgamate any dependencies that have the same left-hand side. Using Result 11.2.1 (2).
5. Repeat steps 2–4 until all the dependencies have been checked.
6. Finally, reduce all the functional dependencies by removing from the right-hand side any attributes on the left-hand side.

Linking schemes

A **relational database** consists of a collection of relation schemes linked together.

We say that there is a **one to many association** between schemes **R** and **S** if it is possible for each tuple in a relation over **R** to be associated with several tuples in a relation over **S**.

A **foreign key** is a set of attributes in one relation scheme **S** that is the primary key of another relation scheme **R**. For there to be a foreign key, **R** and **S** must have a one to many association. We say that the foreign key in the scheme **S** **references** the scheme **R**.

Normal forms

A table is in **first normal form** if it has no duplicate rows and a single-valued entry in each cell.

A relation scheme is in **second normal form** if we **cannot** find the following:

a key K , an attribute x that is not in K , and a proper subset P of K , with $P \rightsquigarrow \{x\}$.

How to move to second normal form

Since the scheme is not in second normal form there will be a set of attributes P that is a proper subset of a key K and that functionally determines some attributes not in K .

Divide the scheme into two schemes.

- The attributes in P together with any attributes that are functionally dependent on P .
- K together with any attributes outside K that are not determined by P .

If either of these schemes is not in second normal form then repeat the process until all the schemes are in second normal form.

In a relation scheme we say that an attribute a is **transitively dependent** on a set of attributes X , where X is a key of the scheme, if there is another set of attributes Y such that

$$X \rightsquigarrow Y \text{ and } Y \rightsquigarrow \{a\} \quad (1)$$

but

$$Y \not\rightarrow X \text{ and } a \notin X \cup Y. \quad (2)$$

We say that a relation scheme is in **third normal form** if it is in second normal form and has no transitive dependencies.

How to move to third normal form

If the scheme is not in third normal form then there will be a transitive dependency $X \rightsquigarrow Y$ and $Y \rightsquigarrow \{a\}$, and where a does not itself determine any other attribute. Identify the smallest subset (say B) of Y that determines a . Form two schemes as follows.

- All the attributes except a .
- The set of attributes B with the attribute a added.

Repeat this process until there are no transitive dependencies.

To be precise, we should add after “ a does not itself determine any other attribute” the comment “except perhaps for an attribute which determines a ”. Any such attribute should be included with B and a when the scheme is divided.

6 RECURRENCE SYSTEMS

As an example, consider a function *INTSUM*, whose input is an integer $n \geq 1$, which returns the sum of the integers between 1 and n .

A recurrence system for *INTSUM* is given below.

$$\text{INTSUM}(1) = 1 \quad (\text{S})$$

$$\text{INTSUM}(n) = \text{INTSUM}(n-1) + n \quad (n > 1) \quad (\text{R})$$

We can use the recurrence system to specify the function formally.

function *INTSUM*(n in Int) **return** in Int

pre $n \geq 1$.

post $\text{INTSUM}(1) == 1$.

$(n > 1) \Rightarrow (\text{INTSUM}(n) == \text{INTSUM}(n-1) + n)$.

A recurrence system leads naturally to recursive code.

```
function INTSUM(n)
{ //recSum
if (n == 1) then
    { return 1 }           //(S)
else
    { return INTSUM(n-1) + n }   //(R)
}
```

For this recurrence system, there is a solution (a formula that gives the same values as the recurrence). To prove that a formula is a solution of a recurrence, we can use **mathematical induction**.

To prove a formula is a solution of a recurrence system

Suppose $U(n)$ is defined in terms of $U(n-1)$ by a recurrence system with an initial condition at $n = M$, that $F(n)$ is given as a formula, and we wish to prove that $U(n) = F(n)$ for all integers $n \geq M$.

1. Show that the formula gives the same value as the initial condition for the recurrence, i.e. show that $U(M) = F(M)$.
2. Show that, for all $k > M$, if the formula gives $U(k-1)$ correctly, then the formula also gives $U(k)$ correctly.

To solve a recurrence system for $T(n)$

1. Use the recursive part of the recurrence repeatedly to write out a suitable set of equations.
2. Write down the starting case at the end of the list of equations.
3. Sum the left-hand and right-hand sides of all these equations. To be useful, the equations should be formed so that most of the terms involving T are the same on each side, and so can be removed.
4. The resulting equation should give $T(n)$ as an expression only involving n . (To obtain a single formula, one may need to use one of the formulae for sums given below.)

Results about sums

$$1 + 2 + 3 + \dots + n = \frac{1}{2}n(n + 1)$$

$$1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{1}{6}n(n + 1)(2n + 1)$$

$$1 + r^1 + r^2 + r^3 + \dots + r^n = \frac{r^{n+1} - 1}{r - 1}, \text{ for any real number } r \neq 1$$

7 EFFICIENCY

For M263, the **time complexity function** of an implementation A is denoted by $T_A(n)$, and gives the total number of statement executions when the implementation is executed for an input of size n in the worst case. The total count of statement executions includes executions of all statements in the code, including **var** and **return** statements, and must take account of the repetition of statements in loops or in recursion. There are some parts of our code that we do not count as statements: lines with only a curly bracket, and comments. The variable n is the size of that feature of the input that principally determines how many statements are executed. The way we measure n will depend on the form of data of this feature — for a sequence it may be its length, for a tree it may be its depth. The number of statement executions may be different for different inputs of the same size — the **worst case** concerns a permitted input of size n that leads to the largest number of statement executions. (The **best case** concerns a permitted input of size n for which the number of statement executions is smallest.)

If one has a formula for the time complexity function, one can classify its behaviour for large values of n by giving the **order** of the function. Some orders that often arise in time complexity are given below.

The time complexity function is sometimes known as the **time efficiency function**.

Hierarchy of time complexity functions

$$\Theta(1), \quad \Theta(\log n), \quad \Theta(\sqrt{n}), \quad \Theta(n), \quad \Theta(n \log n), \quad \Theta(n^2), \quad \Theta(n^3), \quad \Theta(2^n), \quad \Theta(n!)$$

To find the order, $\Theta(F(n))$, of a function $F(n)$ given by a formula:

1. ignore constant multipliers in the formula;
2. find the term in the formula that is furthest to the right in the hierarchy given above;
3. if a function is not given above, use “common sense”.

For a recursive implementation, one can find a recurrence system for the time complexity function. One can use mathematical induction to determine whether or not a proposed formula is a solution of a recurrence system.

8 AXIOMATIC SPECIFICATION

An **axiomatic specification** of a class type gives: **sorts** — the sets involved; **signatures** of the methods; and **axioms** — showing how the methods must interact (their **semantics**). Such an **abstract class** does not include an implementation. An example, **StackC of Int**, is given below.

NAME	StackC of Int
SORTS	Int , the set of integers S , the set of all possible stacks of integers $M = \{ \text{“Error : stack is empty”} \}$
SIGNATURES	$new : \rightarrow S$ (S1) $push : S \times Int \rightarrow S$ (S2) $pop : S \rightarrow S$ (S3) $peek : S \rightarrow Int \amalg M$ (S4) $size : S \rightarrow Int$ (S5)

AXIOMS

$new().size() == 0$	(A1)
$new().peek() == \text{“Error : stack is empty”}$	(A2)
$new().pop() == new()$	(A3)
$\forall s \text{ in } S [\forall elem \text{ in } Int [s.push(elem).size() == s.size() + 1]]$	(A4)
$\forall s \text{ in } S [\forall elem \text{ in } Int [s.push(elem).peek() == elem]]$	(A5)
$\forall s \text{ in } S [\forall elem \text{ in } Int [s.push(elem).pop() == s]]$	(A6)

A **chain** is a sequence of methods applied, one after the other, to an object. The order of application is read from left to right. A chain is **syntactically correct** if it produces a result, i.e. each of the method calls returns the correct form of receiver for the next method call. For example: $new().push(10).push(20)$ is syntactically correct, and results in the stack $[10, 20]$; while $new().size().pop()$ is not syntactically correct, since $new().size()$ yields an integer, which is the incorrect type to be input to pop .

All stack objects of size n are generated from the stack $new()$ by a chain of n applications of the method $push$, as is shown in the chain: $new().push(elem1).push(elem2) \dots push(elemn)$ — this is known as the **canonical form**. We say that the methods new and $push$ are **generators** for **StackC of Int**.

In any chain involving the methods new , pop and $push$, a sub-expression of the form $push(elem).pop()$ can be removed from the chain. Also, the subexpression $new().pop()$ can be replaced by $new()$. The resulting canonical form is called the **reduced form** of the original chain. In the example below, (2) is the reduced form of (1), and is the canonical form for the stack object $[10, 20]$.

$new().pop().push(10).push(30).pop().push(20)$	(1)
$new().push(10).push(20)$	(2)

A set of axioms is said to be **complete** if:
they define the outcome of all permissible applications of the methods of the abstract class;
chains of methods construct all possible instances of the class.

9 TYPE SPECIFICATIONS

Primitive types

Int	Data type
<i>Form of data</i>	Whole numbers: positive, negative or 0.
<i>Type functions</i>	
function (infix) $(x + y \text{ on Int})$ return in Int	
function (infix) $(x \times y \text{ on Int})$ return in Int	(* is used for $\times$ in the WorkPad)
function (infix) $(x - y \text{ on Int})$ return in Int	
function (infix) $(x == y \text{ on Int})$ return in Bool	
function (infix) $(x < y \text{ on Int})$ return in Bool	
function (infix) $(x > y \text{ on Int})$ return in Bool	
function (infix) $(x \leq y \text{ on Int})$ return in Bool	($\leq$ is entered on the keyboard as $<=$)
function (infix) $(x \geq y \text{ on Int})$ return in Bool	($\geq$ is entered on the keyboard as $>=$)
function $MOD(x \text{ in Int}, y \text{ in Int})$ return in Int	
function $DIV(x \text{ in Int}, y \text{ in Int})$ return in Int	

function (infix) $(x + y \text{ on Int})$ **return in Int**

pre true.

post The returned value is the result of adding x and y .

(The binary operations $\times$ and $-$ have similar specifications.)

function (infix) $(x > y \text{ on Int})$ **return in Bool**

pre true.

post The returned value is **true** if the integer x is greater than the integer y and is **false** otherwise.

function (infix) $(x \leq y \text{ on Int})$ **return in Bool**

pre true.

post The returned value is **true** if the integer x is less than or equal to the integer y and is **false** otherwise.

(The comparisons $==$, $\geq$ and $<$ have similar specifications.)

function $DIV(n \text{ in Int}, d \text{ in Int})$ **return in Int**

pre $d > 0$.

post The returned value q is the result of integer division of n by d . It satisfies the condition that, if $r = n - (q \times d)$, then we have $0 \leq r < d$.

function $MOD(n \text{ in Int}, d \text{ in Int})$ **return in Int**

pre $d > 0$.

post The returned value r is the remainder after integer division of n by d . It satisfies the conditions $0 \leq r < d$ and $r = n - (q \times d)$, where q is an integer.

Bool	Data type
<i>Form of data</i>	Values from <i>Bool</i> i.e. true or false .
<i>Type functions</i>	
function <i>NOT</i> (<i>a</i> in Bool) return in Bool	
function (infix) (<i>a</i> $\wedge$ <i>b</i> on Bool) return in Bool	
($\wedge$ is entered on the keyboard as $\wedge$)	
function (infix) (<i>a</i> $\vee$ <i>b</i> on Bool) return in Bool	
($\vee$ is entered on the keyboard as $\vee$)	
function (infix) (<i>a</i> == <i>b</i> on Bool) return in Bool	

function (**infix**)(*a* $\wedge$ *b* on **Bool**) **return in Bool**

pre **true**.

post The returned value is **true** if both *a* is **true** and *b* is **true**, and is **false** otherwise.

function (**infix**)(*a* $\vee$ *b* on **Bool**) **return in Bool**

pre **true**.

post The returned value is **false** if both *a* is **false** and *b* is **false**, and is **true** otherwise.

function (**infix**)(*a* == *b* on **Bool**) **return in Bool**

pre **true**.

post The returned value is **true** if *a* and *b* are the same, and is **false** otherwise.

function *NOT*(*a* in **Bool**) **return in Bool**

pre **true**.

post The returned value is **false** if *a* is **true** and is **true** if *a* is **false**.

Char	Data type
<i>Form of data</i>	Characters from the set <i>Char</i> .
<i>Type functions</i>	
function <i>ASC</i> (<i>c</i> in Char) return in Int	
function <i>CHR</i> (<i>n</i> in Int) return in Char	
function (infix) (<i>c</i> == <i>d</i> on Char) return in Bool	
function (infix)(<i>c</i> < <i>d</i> on Char) return in Bool	
function (infix) (<i>c</i> > <i>d</i> on Char) return in Bool	
function (infix) (<i>c</i> $\leq$ <i>d</i> on Char) return in Bool	
($\leq$ is entered on the keyboard as $\leq$)	
function (infix) (<i>c</i> $\geq$ <i>d</i> on Char) return in Bool	
($\geq$ is entered on the keyboard as $\geq$)	

function *ASC*(*c* in Char) **return** in Int

pre true.

post The returned value is the ASCII code of the character *c*.

See p. 64.

function *CHR*(*n* in Int) **return** in Char

pre $31 < n$ and $n < 127$.

post The returned value is the character with ASCII code *n*.

function (*infix*)(*c* == *d* on Char) **return** in Bool

pre true.

post The returned value is **true** if *ASC*(*c*) == *ASC*(*d*) and is **false** otherwise.

function (*infix*)(*c* < *d* on Char) **return** in Bool

pre true.

post The returned value is **true** if *ASC*(*c*) < *ASC*(*d*) and is **false** otherwise.

(The comparisons $>$, $\leq$ and $\geq$ have similar specifications.)

Array of X

Data type

Form of data An array is a non-empty sequence of elements whose length is fixed. Some places in the array may be blank.

Type functions

function *SIZE*(*arr* in Array of X) **return** in Int

function *AT*(*i* in Int, *arr* in Array of X) **return** in X

function *PUT*(*i* in Int, *arr* in Array of X, *x* in X) **return** in Array of X

function *SIZE*(*arr* in Array of X) **return** in Int

pre true.

post The returned value is the number of items in the sequence *arr*.
Blanks are counted.

function *AT*(*i* in Int, *arr* in Array of X) **return** in X

pre The array *arr* is full and $1 \leq i$ and $i \leq \text{SIZE}(\text{arr})$.

An array is **full** if it has no blanks.

post The returned value is the *i*th item in the sequence *arr*.

function *PUT*(*i* in Int, *arr* in Array of X, *x* in X) **return** in Array of X

pre $1 \leq i$ and $i \leq \text{SIZE}(\text{arr})$.

post The returned value is an array with *x* as its *i*th item, and otherwise identical to *arr*.

Variable declaration: for example, to declare a variable *myArr* of type **Array of Char** and of size 5, write:

var *myArr* in Array (of size 5) of Char

Stack of X	Data type
<i>Form of data</i>	A sequence of elements, whose size may vary.
<i>Default</i>	The sequence is empty.
<i>Type functions</i>	
function <i>SIZE</i> (<i>s</i> in Stack of X) return in Int	
function <i>PUSH</i> (<i>x</i> in X, <i>s</i> in Stack of X) return in Stack of X	
function <i>POP</i> (<i>s</i> in Stack of X) return in Stack of X	
function <i>PEEK</i> (<i>s</i> in Stack of X) return in X	

function *SIZE*(*s* in Stack of X) **return in** Int

pre true.

post The returned value is the number of items in the sequence *s*.

function *PUSH*(*x* in X, *s* in Stack of X) **return in** Stack of X

pre true.

post The returned value is the stack formed by adding *x* to the top of *s*. So, for example, *PUSH*(5, [1, 4, -2]) = [1, 4, -2, 5].

function *POP*(*s* in Stack of X) **return in** Stack of X

pre *s* contains at least one item.

post The returned value is the stack formed by deleting the item from the top of *s*. So, for example, *POP*([1, 4, -2]) = [1, 4].

function *PEEK*(*s* in Stack of X) **return in** X

pre *s* contains at least one item.

post The returned value is the item at the top of *s*. So, for example, *PEEK*([1, 4, -2]) = -2.

Spot types

Spot	Class specification
<i>States</i>	The state of an object <i>mySpot</i> is an ordered pair of integers that defines the position of <i>mySpot</i> on a two-dimensional square grid.
<i>State notation</i>	(<i>X</i> , <i>Y</i>) means that the spot is <i>X</i> places right and <i>Y</i> places up from the default.
<i>Default</i>	The spot is at (0, 0).
<i>Methods</i>	
<i>getXPos</i> () return in Int	<i>downOne</i> () state change
<i>getYPos</i> () return in Int	<i>leftOne</i> () state change
<i>upOne</i> () state change	<i>rightOne</i> () state change

method *getXPos*() **return in** Int

pre true.

post For a spot with state (*X*, *Y*), the returned value is *X*.

method *getYPos()* **return in** **Int**
pre **true**.
post For a spot with state (X, Y) , the returned value is Y .

method *upOne()* **state change**
pre **true**.
post The spot is moved upwards on the grid by one position.

method *downOne()* **state change**
pre **true**.
post The spot is moved downwards on the grid by one position.

method *leftOne()* **state change**
pre **true**.
post The spot is moved left on the grid by one position.

method *rightOne()* **state change**
pre **true**.
post The spot is moved right on the grid by one position.

QuickSpot

Class specification

QuickSpot extends **Spot**

States As for **Spot**.

State notation As for **Spot**.

Default The quickspot is at $(0, 0)$.

Methods

upQuick(n in **Int**) **state change**

downQuick(n in **Int**) **state change**

leftQuick(n in **Int**) **state change**

rightQuick(n in **Int**) **state change**

plus from **Spot**

getXPos, *getYPos*, *upOne*, *downOne*, *leftOne*, *rightOne*

method *upQuick*(n in **Int**) **state change**
pre $n > 0$.
post The quickspot is moved up by n positions.

method *downQuick*(n in **Int**) **state change**
pre $n > 0$.
post The quickspot is moved down by n positions.

method *leftQuick*(n in **Int**) **state change**
pre $n > 0$.
post The quickspot is moved left by n positions.

method *rightQuick*(n in **Int**) **state change**
pre $n > 0$.
post The quickspot is moved right by n positions.

ColourSpot extends Spot

States The state of an object is a position on a two-dimensional square grid (as in **Spot**) together with a colour.

State notation (X, Y, C) means that the spot has position (X, Y) and colour C . The possible colours are: Black, Red, Green and Blue.

Default $(0, 0, \text{Black})$

Methods

getColour() **return in Int**

nextColour() **state change**

plus from **Spot**

getXPos, getYPos, upOne, downOne, leftOne, rightOne

method *getColour()* **return in Int**

pre true.

post The returned value represents the colour of the coloured spot. If the spot is Black, then 0 is returned; if the spot is Red, 1 is returned; if the spot is Green, 2 is returned; if the spot is Blue, 3 is returned.

method *nextColour()* **state change**

pre true.

post The colour of the spot is changed, according to the following order. Black is changed to Red, Red to Green, Green to Blue, and Blue to Black. The position of the spot is not changed.

Turtles

Turtle

Class specification

States An object of type **Turtle** (a turtle) has a state consisting of the following four attributes.

- A **marker**, which itself has the following features.
The position of the marker (a location on the grid, (x, y)).
The direction of the marker (given as “right(0)”, “up(90)”, “left(180)” or “down(270)”).
- A **pen**, which is either “on” or “off”.
- A **diagram**, which consists of all the grid points that the turtle has passed over when its pen state is “on”. (These are stored in a sequence in the order that they were recorded.)
- A **markerStack**, which is a sequence of markers, each being the state of the turtle’s marker at some point in its existence. (At a later stage, the turtle can have its marker state restored to the state of one of the stored markers.)

State notation A 4-tuple giving the states of: the marker, the pen, the diagram and the markerStack.

Default The marker is at the origin, with marker direction “right(0)”. The pen state is “on”. The diagram is empty. The markerStack is empty. So the default state is given by the 4-tuple $((0, 0, \text{“right(0)”}, \text{“on”}, [], [])$.

Methods

<i>forward</i> (<i>n</i> in Int)	state change
<i>leftTurn</i> ()	state change
<i>rightTurn</i> ()	state change
<i>penOff</i> ()	state change
<i>penOn</i> ()	state change
<i>store</i> ()	state change
<i>restore</i> ()	state change

When the pen state is “on”, the grid points passed over by the marker when moving *forward* are added to the diagram. When the pen state is “off”, no record is kept of the grid points as the turtle moves. Think of the diagram as a **set** of spots. These spots are stored in a sequence, but we are not interested in the order in which they are stored, nor in whether a spot has been marked twice.

The only method that requires a precondition other than **true** is *restore*, which requires that the markerStack is not empty. Each method changes the state of the turtle, as described below.

<i>forward</i> (<i>n</i>) (<i>n in Int</i>)	If $n = 0$, the marker state does not change. If the pen is “on” then the current position of the marker is added to the diagram. Hence this single spot will be plotted even though the marker is not moved. If $n > 0$, the marker has its position changed by moving it n places in the current direction. If the pen is “on” then the initial marker position is stored in the diagram, as is each grid point that the marker passes over as it moves forward, including the final marker position. If $n < 0$, the turtle state is not changed.
<i>leftTurn</i> ()	The direction of the marker is changed by turning 90 degrees in an anticlockwise direction.
<i>rightTurn</i> ()	The direction of the marker is changed by turning 90 degrees in a clockwise direction.
<i>penOn</i> ()	The pen state is set to “on”.
<i>penOff</i> ()	The pen state is set to “off”.
<i>store</i> ()	The current marker state of the turtle is added to the markerStack.
<i>restore</i> ()	The turtle’s marker is set to the most recent marker state in the markerStack, and this state is then deleted from the markerStack.

TurtlePlus	Class specification
TurtlePlus extends Turtle <i>State, State notation and Default</i> are as for Turtle. <i>Methods</i> <i>rectangle</i>(<i>side1 in Int, side2 in Int</i>) state change plus all the methods of the class Turtle	

method *rectangle*(*side1 in Int, side2 in Int*) **state change**
pre $side1 \geq 1$ and $side2 \geq 1$ and the receiver has pen state “on”.
post A rectangle is added to the diagram (and the state of the receiver is otherwise unchanged). The first side of the rectangle is drawn by starting at the current marker position of the receiver, and drawing a line of length *side1* in the current marker direction. The next side is drawn after first turning 90 degrees anticlockwise, and has length *side2*. (Then draw two more sides to complete the rectangle.)

SuperTurtle

Class specification

SuperTurtle extends TurtlePlus

State, State notation and Default are as for **Turtle**.

Methods

diagonal(n in Int) **state change**

plus *rectangle* from **TurtlePlus**, and all the methods of **Turtle**

method *diagonal*(n in Int) **state change**

pre $n \geq 1$ and the receiver has pen state “on”.

post The state of the receiver is changed in two ways.

A diagonal line is added to the diagram. The diagonal line is drawn as follows. Turn 45 degrees anticlockwise from the current marker direction. Then add a diagonal line of $n + 1$ spots in that direction (including spots at the current marker position and the final marker position).

The marker position is changed to the final point on this diagonal line.

Marker

Class specification

Marker extends Spot

States A marker has a position on a two-dimensional grid (as in **Spot**) together with a direction.

State notation (x, y, n) where x is the x -coordinate of the marker, y is the y -coordinate of the marker and n is the direction of the marker.

Default The marker is in the state $(0, 0, \text{“right(0)”})$.

Methods

nextDirection() **state change**

plus from **Spot**

getXPos, getYPos, upOne, downOne, leftOne, rightOne

method *nextDirection*() **state change**

pre **true**.

post The marker’s direction attribute is changed so as to rotate the marker 90 degrees anticlockwise. (So “right(0)” is changed to “up(90)”, “up(90)” to “left(180)”, “left(180)” to “down(270)” and “down(270)” to “right(0)”.)

Sequence based types

Vector of X	Class specification
<i>States</i>	The state of an object is a sequence of items from X .
<i>State notation</i>	The state is shown in the usual sequence notation.
<i>Default</i>	The empty sequence, <code>[]</code> .
<i>Methods</i>	
<i>setVector(list of X)</i>	state change
<i>size()</i>	return in Int
<i>addFirst(x in X)</i>	state change
<i>addLast(x in X)</i>	state change
<i>setAt(n in Int, x in X)</i>	state change
<i>removeFirst()</i>	state change
<i>removeLast()</i>	state change
<i>isEmpty()</i>	return in Bool
<i>contains(x in X)</i>	return in Bool
<i>first()</i>	return in X
<i>last()</i>	return in X
<i>at(n in Int)</i>	return in X

method *setVector(list of X)* state change

pre true.

post The state of the receiver object becomes the sequence given in *list*.

The parameter *list* represents a sequence of values of type **X**. For example, for *v* of type **Vector of Int**, the statement *v.setVector(4, 5, 7)* will set its state to `[4, 5, 7]`. The parameter *list* must be provided as a list of specific values from **X**. An array or stack or vector of integers will not be regarded as a list of integers in this context.

method *size()* return in Int

pre true.

post The returned value is the number of elements in the receiver.

method *addFirst(x in X)* state change

pre true.

post Changes the state of the receiver by adding the item *x* to the front.

method *addLast(x in X)* state change

pre true.

post Changes the state of the receiver by adding the item *x* to the back.

method *setAt(n in Int, x in X)* state change

pre $1 \leq n$ and $n \leq$ the size of the receiver.

post Changes the state of the receiver by placing the element *x* in the *n*th position (replacing whatever was there before).

method *removeFirst*() **state change**

pre The receiver is not empty.

post Changes the state of the receiver by deleting its first element.

method *removeLast*() **state change**

pre The receiver is not empty.

post Changes the state of the receiver by deleting its last element.

method *isEmpty*() **return in Bool**

pre **true**.

post The returned value is **true** if the receiver is empty, and is **false** otherwise.

method *contains*(*x* in **X**) **return in Bool**

pre **true**.

post The returned value is **true** if the item *x* is an element of the receiver, and is **false** otherwise.

method *first*() **return in X**

pre The receiver is not empty.

post The returned value is the first element in the receiver.

method *last*() **return in X**

pre The receiver is not empty.

post The returned value is the last element in the receiver.

method *at*(*n* in **Int**) **return in X**

pre $1 \leq n$ and $n \leq$ the size of the receiver.

post The returned value is the element in the *n*th position in the receiver.

VectorPlus of X

Class specification

VectorPlus of X extends Vector of X

States, State notation and *Default* are as for **Vector**.

Methods

remove(*n* in **Int**) **state change**

insert(*n* in **Int**, *x* in **X**) **state change**

sort() **state change**

join(*v* in **Vector of X**) **state change**

plus all the methods of **Vector**

We assume that **X** includes a comparison. For example, **X** might be **Int** or **Char**.

method *remove*(*n* in **Int**) **state change**

pre $1 \leq n$ and $n \leq$ the size of the receiver.

post Changes the state of the receiver by removing the element with index *n*.

method *insert*(*n* in Int, *x* in X) **state change**

pre $1 \leq n$ and $(n \leq 1 + \text{the size of the receiver})$.

post Changes the state of the receiver by inserting the element *x* so that it becomes the element with index *n*.

method *sort*() **state change**

pre **true**.

post Changes the state of the receiver by sorting its elements into ascending order.

method *join*(*v* in Vector of X) **state change**

pre **true**.

post Changes the state of the receiver by joining *v* to the end of the receiver.

mString	Class specification
<i>States</i>	A sequence of characters (a string).
<i>State notation</i>	The state is shown using the double inverted comma notation.
<i>Default</i>	The empty string (written "").
<i>Methods</i>	
<i>setString</i> (<i>listOfChar</i>) state change	
<i>length</i> () return in Int	
<i>charAt</i> (<i>i</i> in Int) return in Char	
<i>substring</i> (<i>n</i> in Int, <i>m</i> in Int) return in mString	
<i>substr</i> (<i>n</i> in Int, <i>len</i> in Int) return in mString	
<i>charAppend</i> (<i>c</i> in Char) return in mString	
<i>concat</i> (<i>s</i> in mString) return in mString	
<i>equal</i> (<i>s</i> in mString) return in Bool	
<i>hashCode</i> () return in Int	

method *setString*(*listOfChar*) **state change**

pre **true**.

post The state of the receiver becomes the sequence of characters given as *listOfChar*.

The parameter *listOfChar* is given in a form such as "Querty".

method *length*() **return in Int**

pre **true**.

post The returned value is the number of characters in the receiver.

method *charAt*(*n* in Int) **return in Char**

pre $1 \leq n$ and $n \leq \text{the length of the receiver}$.

post The returned value is the character with index *n* in the receiver.

A **substring** of a string is a continuous run of characters taken from that string. For example “wert”, “tyu” and “q” are all substrings of the string “qwertyu”.

method *substring*(*n in Int, m in Int*) **return in mString**

pre $1 \leq n$ and $n \leq m$ and $m \leq$ the length of the receiver.

post The returned value is a string consisting of a substring of characters taken from the receiver. This substring starts with the character at index position *n* and finishes with the character at index position *m*, including each of these.

method *substr*(*n in Int, len in Int*) **return in mString**

pre $1 \leq n$ and $n \leq$ the length of the receiver and

$0 \leq len$ and $(n + len - 1) \leq$ the length of the receiver.

post The returned value is a string consisting of a substring of characters taken from the receiver. This substring has exactly *len* characters, and starts with the character at the index position *n* in the receiver. If *len* is 0, then the returned string is empty.

method *charAppend*(*c in Char*) **return in mString**

pre **true**.

post The returned value is a string consisting of the receiver with the character *c* added to the end.

method *concat*(*s in mString*) **return in mString**

pre **true**.

post The returned value is a string that has the string *s* concatenated to the end of the receiver.

method *equal*(*s in mString*) **return in Bool**

pre **true**.

post The returned value is **true** if the receiver and the string *s* are identical (i.e. have the same length and contain the same characters in the same order), and is **false** otherwise.

For *concat* and *equal*, the parameter must be another object of type **mString**, and **not** a list of characters written between inverted commas.

method *hashCode*() **return in Int**

pre **true**.

post The returned value is a number associated with the first three characters of the receiver. See Unit 7, p. 43.

Pair of X,Y	Class specification
<i>States</i>	The state of an object is an ordered pair of elements (x,y) , where x is in X and y is in Y .
<i>State notation</i>	(x,y) .
<i>Default</i>	There is no default.
<i>Methods</i>	
	<i>setPair</i> (x in X , y in Y) state change

method *setPair*(x in X , y in Y) state change

pre true.

post The state of the receiver is set to (x,y) .

As there is no default state, a variable of type **Pair** must be initialised using the method *setPair*.

Sets

Set of X	Class specification
<i>States</i>	The state of an object is a set of elements of type X .
<i>State notation</i>	A list of elements enclosed in curly brackets and separated by commas.
<i>Default</i>	The empty set $\{\}$.
<i>Methods</i>	
	<i>setSet</i> (<i>list</i> of X) state change
	<i>addElement</i> (<i>element</i> in X) state change
	<i>card</i> () return in Int
	<i>contains</i> (<i>element</i> in X) return in Bool
	<i>containsSet</i> (A in Set of X) return in Bool
	<i>equals</i> (A in Set of X) return in Bool
	<i>union</i> (A in Set of X) return in Set of X
	<i>intersect</i> (A in Set of X) return in Set of X
	<i>minus</i> (A in Set of X) return in Set of X
	<i>cartProd</i> (A in Set of Y) return in Set of Pair of X , Y

method *setSet*(*list* of **X**) state change

pre true.

post The state of the receiver becomes the set derived from *list* by enumeration.

The parameter *list* is a list of items of type **X**, and the resulting state is the set derived from the list by enumeration — any repeated values in the list are ignored. For example, the statement $s.setSet(2, 1, 3, 3, 1, 2)$ would change the state of the object s to $\{2, 1, 3\}$.

method *addElement*(*element* in **X**) **state change**

pre **true**.

post The element x is added to the receiver (if it is not already there).

The two methods above are the only state-changing methods of the class **Set**. The other methods return values, as summarised below.

Method	Returned value
<i>card</i>	The cardinality of R
<i>contains</i> (<i>element</i>)	$element \in R$
<i>containsSet</i> (A)	$A \subseteq R$
<i>equals</i> (A)	$R == A$
<i>union</i> (A)	$R \cup A$
<i>intersect</i> (A)	$R \cap A$
<i>minus</i> (A)	$R - A$
<i>cartProd</i> (A)	$R \times A$

R denotes the receiver set in all these specifications.

method *card*() **return in Int**

pre **true**.

post The returned value is the number of elements in R (its cardinality).

method *contains*(*element* in **X**) **return in Bool**

pre **true**.

post The returned value is **true** if R contains *element* as a member, and is **false** otherwise.

method *containsSet*(A in **Set of X**) **return in Bool**

pre **true**.

post The returned value is **true** if A is a subset of R , and is **false** otherwise.

method *equals*(A in **Set of X**) **return in Bool**

pre **true**.

post The returned value is **true** if A and R are equal sets, and is **false** otherwise.

method *union*(A in **Set of X**) **return in Set of X**

pre **true**.

post The returned value is the set $R \cup A$.

method *intersect*(A in **Set of X**) **return in Set of X**

pre **true**.

post The returned value is the set $R \cap A$.

method *minus*(A in **Set of X**) **return in Set of X**

pre **true**.

post The returned value is the set $R - A$.

method *cartProd*(A in **Set of X**) **return in Set of Pair of X,Y**

pre **true**.

post The returned value is the set $R \times A$.

Binary trees

BinTree of X	Class specification
<i>States</i>	The state of an object is a binary tree, which has a data item from X stored at each node.
<i>State notation</i>	The state of an object will usually be shown graphically, but we can use the linear notation.
<i>Default</i>	The empty tree.
<i>Methods</i>	
<i>setTree</i> (<i>item</i> in X , <i>l</i> in BinTree , <i>r</i> in BinTree)	state change
<i>setRoot</i> (<i>item</i> in X)	state change
<i>setLeftTree</i> (<i>newLeft</i> in BinTree)	state change
<i>setRightTree</i> (<i>newRight</i> in BinTree)	state change
<i>getRoot</i> ()	return in X
<i>getDepth</i> ()	return in Int
<i>isEmpty</i> ()	return in Bool
<i>leftTree</i> ()	return in BinTree
<i>rightTree</i> ()	return in BinTree

In the signatures, **BinTree** of **X** is shortened to **BinTree**.

method *setTree*(*item* in **X**, *l* in **BinTree**, *r* in **BinTree**) **state change**

pre true.

post Changes the state of the receiver to a binary tree whose root is *item*, whose left subtree is *l* and whose right subtree is *r*.

method *setRoot*(*item* in **X**) **state change**

pre true.

post If the receiver is an empty tree, then its state is changed to a tree whose root is *item*, and whose left and right subtrees are empty. If the receiver is not empty, then its state is changed by changing the root element to *item*, with the left and right subtrees remaining unchanged.

method *setLeftTree*(*newLeft* in **BinTree**) **state change**

pre The receiver is not empty.

post Changes the state of the receiver by changing the left subtree to *newLeft*. The root and the right subtree are unchanged.

method *setRightTree*(*newRight* in **BinTree**) **state change**

pre The receiver is not empty.

post Changes the state of the receiver by changing the right subtree to *newRight*. The root and the left subtree are unchanged.

method *getRoot*() **return in X**

pre The receiver is not empty.

post The returned value is the item stored at the root node in the receiver.

method *getDepth*() **return in Int**

pre true.

post The returned value is the depth of the receiver (i.e. the returned value is 0 for an empty tree; the number of levels for a non-empty tree).

method *isEmpty()* **return in Bool**

pre true.

post The returned value is **true** if the receiver is the empty tree and is **false** otherwise.

method *leftTree()* **return in BinTree**

pre true.

post If the receiver is not empty, then its left subtree is returned. If the receiver is an empty tree, then the returned value is an empty tree.

method *rightTree()* **return in BinTree**

pre true.

post If the receiver is not empty, then its right subtree is returned. If the receiver is an empty tree, then the returned value is an empty tree.

Class implementations

The classes **Spot**, **QuickSpot** and **ColourSpot** are available in the M263 class library. There is no need to enter implementations for these classes into the WorkPad. We give their implementations here to illustrate how the implementation of a new class can be expressed in a form suitable for the WorkPad.

Spot

```
Class Spot
{
  this.xPos in Int
  this.yPos in Int
  /* These statements show the attributes used to
  implement the data set of Spot. */
  this.xPos <-- 0
  this.yPos <-- 0
  /* These two statements give the implementation of the
  default state. */
  this.getXPos calls getXPosSpot
  this.getYPos calls getYPosSpot
  this.upOne calls upOneSpot
  this.downOne calls downOneSpot
  this.leftOne calls leftOneSpot
  this.rightOne calls rightOneSpot
  /* These statements show which code should be used
  when a method is called. */
}

Spot extends Base

/* All WorkPad classes extend Base, either directly
or indirectly. This makes clone and show available. */
```

```

method    getXPosSpot( )
{
return this.xPos
}
method    getYPosSpot( )
{
return this.yPos
}
method    upOneSpot( )
{
this.yPos <-- this.yPos + 1
}
method    downOneSpot( )
{
this.yPos <-- this.yPos - 1
}
method    leftOneSpot( )
{
this.xPos <-- this.xPos - 1
}
method    rightOneSpot( )
{
this.xPos <-- this.xPos + 1
}

```

QuickSpot

```

Class QuickSpot
{
this.upQuick calls upQuickQuickSpot
this.downQuick calls downQuickQuickSpot
this.leftQuick calls leftQuickQuickSpot
this.rightQuick calls rightQuickQuickSpot
}

```

QuickSpot extends Spot

```

method    upQuickQuickSpot(n)
{
var index in Int
for (index <-- 1 to n)
{ this.upOne( ) }
}

```

```

method    downQuickQuickSpot(n)
{
this.yPos <-- this.yPos - n
}

```

```

method leftQuickQuickSpot(n)
{
  var index in Int
  for (index <-- 1 to n)
  {
    this.leftOne( )
  }
}

```

```

method rightQuickQuickSpot(n)
{
  this.xPos <-- this.xPos + n
}

```

The default state is inherited from **Spot**. The implementations of *getXPos*, *getYPos*, *upOne*, *downOne*, *leftOne* and *rightOne* are all inherited from **Spot**. Different styles of implementation are given for *leftQuick* and *upQuick* in contrast to *downQuick* and *rightQuick* to illustrate two possible approaches. Either style of implementation could be used for all four methods here.

ColourSpot

```

Class ColourSpot
{
  this.colour in Int
  /* The attributes xPos and yPos are automatically
  inherited from Spot by extension. This statement
  shows that ColourSpot also has the additional
  attribute colour. */
  this.colour <-- 0
  /* Gives the default value of the attribute colour. */
  this.getColour calls getColourColourSpot
  this.nextColour calls nextColourColourSpot
}

ColourSpot extends Spot

```

The default values for xPos and yPos are inherited from Spot.

```

method getColourColourSpot( )
{
  return this.colour
}

```

```

method nextColourColourSpot( )
{
  if (this.colour == 3) then
  {this.colour <-- 0}
  else {this.colour <-- this.colour + 1}
}

```

10 GLOSSARY

Absorption laws: $s \vee T \equiv T$; $s \vee F \equiv s$; $s \wedge T \equiv s$; $s \wedge F \equiv F$.	Unit 10
abstract data type (or ADT): A type whose specification is independent of its implementation.	Unit 16
abstraction: Separation of the specification (a statement of behaviour) and the implementation (code to achieve that behaviour).	Unit 2
algorithm: A strategy for solving a problem, such as bubble sort or heap sort.	Units 6, 15
alternate key: When there is a one-to-one association between relation schemes there are two possible choices of foreign key. We use the term alternate key in this case.	Unit 11
argument (logical): An argument is an assertion that a set of propositions $s_1, s_2, \dots, s_n$, yields another proposition t .	Unit 10
argument (of a predicate): The word argument is often used for the input(s) to a predicate (or those of a function).	Unit 10
arity of a predicate: Each predicate has an associated arity, which is the number of input values to which it applies.	Unit 10
array: A variable of type Array of X is referred to as an array. The form of data of an array is a non-empty sequence (of elements from X) whose size is fixed. All the items in an array will be of the same type. An array variable has access to the type operations <i>SIZE</i> , <i>PUT</i> and <i>AT</i> .	Unit 3
ASCII code: Each character in the set <i>Char</i> has an associated ASCII code, which is an integer between 32 and 126 (inclusive).	Unit 1
assignment statement: An assignment statement $current \leftarrow something$ first calculates the expression, <i>something</i> , that appears to the right of $\leftarrow$, using (if necessary) the value stored in <i>current</i> before the statement is executed, and then stores the calculated value of <i>something</i> in the storage location <i>current</i> .	Unit 2
assignment symbol: The assignment symbol is the symbol $\leftarrow$ used in an assignment statement (in the course code). In code for the WorkPad, the assignment symbol is replaced by $<--$.	Unit 2
Associative laws: $s \vee (t \vee r) \equiv (s \vee t) \vee r$; $s \wedge (t \wedge r) \equiv (s \wedge t) \wedge r$.	Unit 10
asymptotic behaviour (of a function): The behaviour of a function $f(n)$ for large values of n .	Unit 15
asymptotic domination: For functions f and g taking positive values, g asymptotically dominates f if there are positive numbers c and N such that $f(n) \leq c \times g(n)$ for all $n \geq N$.	Unit 15
atom: In a compound Boolean expression such as $a \wedge (b \vee \neg c)$, we call the Boolean variables a , b and c atoms.	Unit 9
attribute of a class: The data set of a class is implemented using attributes. The state of an object is determined by the current value stored for each of the attributes of its class.	Unit 4

attribute of a relation scheme: The headings of a relation table are known as the attributes of the data.	Unit 11
average case (for time complexity): The “average case” for time complexity will take account of the time cost of different possible inputs of the same size, and, ideally, the frequency with which they will occur in practice.	Unit 15
axiom: In the formal axiomatic specification of an ADT, an axiom shows how the methods (or type functions) should relate to each other, and will be expressed as a proposition that is always true .	Unit 16
balanced tree: A tree is balanced if, below each node, the left and right branches lead to subtrees whose depths do not differ by more than 1.	Unit 8
Base: When implementing a new class (say MyClass) in the Work-Pad, a statement MyClass extends Base must be included (in order to make <i>show</i> and <i>clone</i> available to MyClass). (If MyClass is an extension of an existing class, then this is not needed, since MyClass will extend Base indirectly.)	Unit 4
basis for induction: Another term for the starting step in an inductive proof.	Unit 14
best case (for time complexity): In determining the time complexity $T(n)$, the best case relates to an input of size n for which the total number of statement executions is least.	Unit 15
big Oh: For functions f and g , $f(n)$ is $O(g(n))$ if f is asymptotically dominated by g .	Unit 15
binary notation (for an integer): A method of expressing an integer that is based on the number 2.	Unit 3
binary operation: A function whose input set is of the form $X \times X$. Addition of integers is an example of a binary operation. We often use infix notation for binary operations (but not always).	Unit 1
binary search: To search for an item in a BST, we need only to check the root, then search in the appropriate subtree. We can use a similar approach on a sorted sequence. Such searches are significantly more efficient than a search in which every item in the collection must be checked.	Units 6, 8, 13, 15
binary search tree: A binary search tree (or BST) is a sorted tree in which the data items appear so that they increase in size as one goes through the tree from left to right. Formally, a non-empty tree is a BST if: the value at the root is greater than (or equal to) every value in its left subtree, and is less than (or equal to) every value in its right subtree; and its left subtree and its right subtree are themselves both binary search trees.	Unit 8
binary tree: A binary tree may be empty, but if not empty it has a root, a left subtree and a right subtree (where these subtrees may be empty). In tree diagrams, the root is shown at the top, with left and right branches leading to the two subtrees (where necessary) — empty subtree are not shown.	Unit 8

binding term: The binding term consists of the quantifier, variable and set to which the variable belongs.	Unit 10
blank (in an array): After being created by a var statement, an array variable contains no data, so all its places are blank. The array may be filled by assignment of an array literal, or by repeated use of <i>PUT</i> . In traces, blanks in arrays are shown as $\perp$.	Unit 3
body of a loop: The statements to be repeated in a for or while loop form the body of the loop. The statements are enclosed in curly brackets: { and }.	Unit 2
<i>Bool</i> : The set of truth (or Boolean) values: { true , false }.	Unit 1
Boolean algebra: An algebra, where the terms are Boolean expressions, with laws that allow for the rearrangement of these expressions.	Unit 10
Boolean expression: An expression involving variables that represent values in <i>Bool</i> , and operations combining such values.	Unit 9
Boolean search: A form of search using Boolean expressions.	Unit 9
Boolean variable: A variable whose value lies in the set <i>Bool</i> , i.e. a variable whose value is either true or false .	Unit 9
bound variable: A variable that is restricted by a quantifier is known as a bound variable.	Unit 10
bracket counting: When forming a parse tree, we can determine the root node by counting brackets, and recording the count as each operation is reached. The root node will be the operation where the bracket count is zero.	Unit 9
branch (of a tree): In a tree, nodes are connected by branches, shown as lines in a tree diagram.	Unit 8
BST: See binary search tree.	Unit 8
bubble sort: An algorithm for sorting a sequence into ascending order, based on comparison of adjacent pairs, with the pair being exchanged if necessary.	Unit 6
calls: When providing the implementation of a new class (say My-Class) in the WorkPad, separate identifiers are given for a method and for the code that implements it. When a method (say <i>aMethod</i>) is called, a statement of the form aMethod calls aMethodMyClass shows the code (identified as <i>aMethodMyClass</i>) to be used.	Unit 4
candidate key: Where a relation scheme has more than one key, the keys of the scheme may then be referred to as candidate keys.	Unit 11
cardinality: The number of elements in a finite set. (Remember to ignore any repeated elements when finding the cardinality of a set given as a list.)	Unit 7
Cartesian product: The Cartesian product of sets <i>X</i> and <i>Y</i> is written as $X \times Y$. It consists of all the ordered pairs (or 2-tuples) of the form (x, y) where <i>x</i> is from the set <i>X</i> and <i>y</i> is from the set <i>Y</i> . For example, the ordered pair (2, 'Q') comes from $Int \times Char$.	Units 1, 7
case sensitive: Used of programming languages that distinguish between upper- and lower-case letter characters in identifiers.	Unit 3

casting: A technique available in some programming languages for moving values between types; this is not available in the course code.	Unit 6
<i>Char:</i> The set of characters considered in the course (with ASCII codes between 32 and 126 inclusive).	Unit 1
character: A symbol that can be typed from a computer keyboard (possibly with the Shift key pressed). Character literals are written in single inverted commas, as in: 'Q'. In M263, we only consider characters with ASCII codes between 32 and 126 inclusive.	Unit 1
child node (in a tree): If two nodes in a tree are connected by a branch, the node nearer to the root is called the parent, and the node further from the root is called the child.	Unit 8
class: A data type whose type functions are given as methods (including an implementation in terms of available types).	Unit 4
class extension: A relationship between classes. Consider two classes, A and B say. If A extends B , then the following are true. A has all the methods of B , and A has all the attributes of B . Class A may have methods that B does not have, and A may have attributes that B does not have. In the course code, extension should not change the effect of inherited methods. If A extends B , then any method of class B is available to the class A , and its effect in A must be consistent with its effect in B .	Unit 4
class implementation: The implementation of a class must give: (i) an implementation of the data set of the class, as attributes each of which has a type that is an existing data type (either a primitive type or a class that has already been implemented), and (ii) an implementation of each of the class methods. Some of these features may be inherited through an extends relationship with an existing class.	Unit 4
class specification: A description of the behaviour of a class (but not of how this is achieved). It consists of a description of the possible states of an instance of the class, and of the default state of the class, together with specifications of each of the class methods.	Unit 4
clone: In the course code, assignment between objects (variables of class type) is only permitted in the form $a \leftarrow b.clone()$. This ensures that the effect of the assignment is to change the state of a to become identical to the current state of b , without establishing any permanent link between the objects a and b .	Unit 4
collection type: Any type whose form of data is a collection of elements of a particular type, such as a sequence (say of integers) or a binary tree (say of strings).	Unit 6
comment: Comments are included in code to explain what the code is doing. Comments are not statements addressed to the computer, which cannot interpret them and will just ignore them, but rather to a person trying to follow the code. Comments may be included in code between the delimiters <code>/*</code> and <code>*/</code> or (when on a single line) preceded by <code>//</code> .	Unit 2

Commutative laws: $s \vee t \equiv t \vee s$; $s \wedge t \equiv t \wedge s$. Unit 10

Commutativity for quantifiers:

$\forall x \text{ in } U[\forall y \text{ in } V[p(x, y)]] \equiv \forall y \text{ in } V[\forall x \text{ in } U[p(x, y)]]$;
 $\exists x \text{ in } U[\exists y \text{ in } V[p(x, y)]] \equiv \exists y \text{ in } V[\exists x \text{ in } U[p(x, y)]]$. Unit 10

complement (of a set): If A is a set drawn from a predefined set U , then $U - A$ is called the complement of A . Unit 7

complete information: If we know the truth value of every possible proposition about a domain of interest, we have complete information about it. Unit 10

complete sum-of-products form: Another term for disjunctive normal form. Unit 9

complete tree: A complete tree has all but one of its levels completely filled. Also, the bottom level in the tree is being filled in a systematic way, working from the left. Unit 8

completely filled: In a non-empty binary tree, we say that a level is completely filled if there are as many nodes at that level as there possibly can be. Unit 8

component: In an ordered pair (x, y) , x is the first component and y is the second component. Unit 7

composite proposition: Suppose we have some simple propositions whose interpretation in English has been given. We can use operations such as $\vee$ or $\Rightarrow$ to combine these to form new propositions. We refer to such a combination as a composite proposition. Unit 9

composition of classes: In the implementation of a class, the use of an existing class as the type of an attribute is known as composition of classes. Class composition enables methods to be “hidden”, which class extension does not Unit 5

computationally hard: If all known algorithms to solve a problem perform badly for relatively large problem sizes, we say the problem is computationally hard. Unit 15

concatenation: Merging two sequences to form a single sequence. To concatenate sequences s and t , first write the members of s , then the members of t . (The resulting sequence is often written as $s \oplus t$.) Unit 3

conclusion: In an argument corresponding to an expression

$$(s_1 \wedge s_2 \wedge \dots \wedge s_n) \Rightarrow t,$$

we call t the conclusion of the argument. Unit 10

conditional statement: A statement of the general form

if (condition) **then** {codeOne} **else** {codeTwo}

(where **else** may be omitted). The statement

if (condition) **then** {codeOne}

performs codeOne if the condition is **true**. If the condition is **false**, then codeOne is not performed. The statement

if (condition) **then** {codeOne} **else** {codeTwo}

performs codeOne if the condition is **true**, and performs codeTwo if the condition is **false**.

Unit 2

conjunction: An alternative name for the binary operation $\wedge$ (and) on *Bool*.

Unit 1

consistent: A set of propositions $p_1, p_2, \dots, p_n$ is called consistent if it is possible for $p_1 \wedge p_2 \wedge \dots \wedge p_n$ to be **true**.

Unit 10

constant polynomial: A polynomial consisting of just a constant term (so its value does not vary).

Unit 15

contradiction: A Boolean expression is a contradiction if it is equivalent to **false**.

Unit 10

contrapositive rule: This is the rule
$$\frac{s \Rightarrow t; \neg t}{\neg s}.$$

Unit 10

correct (implementation of a function): For a given input, the value returned by an implementation of a function is correct if it satisfies the postcondition in the specification. An implementation of a function specification is correct if the correct value is returned for all value(s) of the type(s) given in the signature that satisfy the precondition in the specification.

Unit 2

course code: A language and notation that we use to give statements expressed in a way suitable for interpretation by a computer. This code is peculiar to this course. It includes forms of statement that are typical of programming languages of a particular style.

Unit 2

cubic polynomial: A polynomial (in n) is cubic if its term with the highest power of n is a constant multiple of n^3 .

Unit 15

data duplication: Data duplication is where data is kept in a number of different tuples.

Unit 11

data set: For a given data type (primitive type or class type), the data set is a set giving the possible states of a variable of that type.

Unit 3

data type: A form of data together with processes that may be applied to data of that form. Declaring the data type of a variable tells you what processes are available to that variable.

Unit 3

decimal notation: The usual notation for writing integers (which is based on the number 10).

Unit 3

default value: If a data type has a default value, then, when a variable of that type is declared, it is created with the default value.

Unit 3

De Morgan's laws: $\neg(s \vee t) \equiv \neg s \wedge \neg t$; $\neg(s \wedge t) \equiv \neg s \vee \neg t$.	Unit 10
De Morgan's laws (for quantifiers): $\neg \forall x \text{ in } U[p(x)] \equiv \exists x \text{ in } U[\neg p(x)]$; $\neg \exists x \text{ in } U[p(x)] \equiv \forall x \text{ in } U[\neg p(x)]$.	Unit 10
De Morgan's laws (for sets): $U - (A \cup B) = (U - A) \cap (U - B)$; $U - (A \cap B) = (U - A) \cup (U - B)$.	Unit 10
degree (of a polynomial): A polynomial in n is a sum of terms of the form $c \times n^r$. The degree of the polynomial is the largest value of the power r in any of its terms.	Unit 15
depth (of a tree): The number of levels in a tree. (The empty tree has depth 0.)	Unit 8
derivation by transitivity: If $A \rightsquigarrow B$ and $B \rightsquigarrow C$ then $A \rightsquigarrow C$.	Unit 11
determining property: We say that any attribute or set of attributes that determines all the other attributes of a scheme has the determining property.	Unit 11
diagram (of a turtle): The attribute of an object of type Turtle that records the spots that have been marked.	Unit 5
disjoint sets: Sets with nothing in common (i.e. whose intersection is the empty set) are called disjoint.	Unit 7
disjoint union: The disjoint union of sets X and Y , written $X \amalg Y$, consists of all items that come either from X or from Y (where X and Y have nothing in common).	Unit 1
disjunction: An alternative name for the binary operation $\vee$ (or) on <i>Bool</i> .	Unit 1
disjunctive normal form: The disjunctive normal form of an expression contains one term for each row of its truth table that evaluates to true . These terms are connected by $\vee$.	Unit 9
Distributive laws: $s \vee (t \wedge r) \equiv (s \vee t) \wedge (s \vee r)$; $s \wedge (t \vee r) \equiv (s \wedge t) \vee (s \wedge r)$	Unit 10
divisor: In integer division of n by d , the second integer, d , is known as the divisor.	Unit 1
domain of a function: The set of values in the input set of a function that satisfy its precondition.	Unit 1
domain of an attribute: Each attribute in a relation scheme has an associated data set called the domain of the attribute.	Unit 11
domain of interest: This is a predefined set, or collection of predefined sets, to which all the objects under discussion belong.	Unit 10
dot notation: The notation used in a statement to show application of a method to an object.	Unit 4
dynamic growth: A type whose form of data can change in size, such as Stack or Vector , is said to have dynamic growth.	Unit 6
element (of a sequence): The items appearing in a sequence may be referred to as the elements of the sequence.	Unit 1

element (of a set): The items in a set may be referred to as the elements of the set.	Unit 1
empty sequence: A sequence with no elements. It has length 0 and is written as $[]$.	Unit 1
empty set: A set containing no items. It has cardinality 0 and is written as $\{\}$.	Unit 1
empty tree: A tree with no nodes. It has depth 0, and is written as $()$ in the linear notation, and as $ET()$ in code.	Unit 8
equivalence rule: This is the rule $\frac{s \Leftrightarrow t; s}{t}$.	Unit 10
equivalent: In logic, a proposition $p \Leftrightarrow q$ can be read as “ p is equivalent to q ”, or as “ p if and only if q ”.	Unit 9
equivalent (Boolean expressions): Two Boolean expressions are said to be equivalent if their values are always the same whatever the values of the variables in the expressions.	Unit 9
existential quantifier: The existential quantifier, which is denoted by $\exists$, is literally read as “there exists”. Expressions of the same idea include “there is”, “for some” or “there is some”. The existential quantifier precedes a variable that must come from a specified set in the domain of interest.	Unit 10
explicit functional dependency: Explicit functional dependencies of a relation scheme are those that follow directly from the rules of the scheme.	Unit 11
extends: A reserved word of the course code signalling class extension.	Unit 4
extension: See class extension.	Unit 4
factor: The integer n is a factor of the integer m if it divides it exactly, so that the remainder when m is divided by n is 0.	Unit 3
factorial: For an integer $n \geq 1$, the product of the integers between 1 and n is written $n!$ (read as “ n factorial”).	Unit 13
finite sequence: A sequence with an end is called finite (in contrast with a sequence such as $1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \dots$, which has a member for every positive integer, and has no end).	Unit 1
floating point: A real number given in a form such as 0.4355×10^3 .	Unit 1
foreign key: A foreign key is a set of attributes in one relation scheme R that is the primary key of another relation scheme S . For there to be a foreign key, S and R must have a one to many association. We say that the foreign key in the scheme R references the scheme S .	Unit 11
form of data: A general term used in the course to reflect differences in the nature of data, be it primitive (unstructured) data, such as integers or characters, or more structured forms, such as sequences, sets or trees.	Unit 1
formal specification: A specification expressed in mathematical notation, such as that of logic (or in a formal language, such as VDM or Z).	Units 10, 16

free variable: If an expression contains a variable and that variable is not restricted by a quantifier, we call the variable a free variable.	Unit 10
full array: An array with no blanks.	Unit 3
full binary tree: A binary tree with all its levels completely filled (so that it has the greatest possible number of nodes for a given depth).	Unit 14
full description (of a function): A function description with lines giving: the signature line; the precondition; the postcondition.	Unit 1
function: A process that, when given an input of a specified type and satisfying a given precondition, yields a unique output.	Unit 1
functionally dependent: We say that the set of attributes Y is functionally dependent on the set of attributes X , and write $X \rightsquigarrow Y$, if the values of the attributes in Y are known whenever we know the values of all the attributes in X .	Unit 11
generic type: Another term for a parameterised type.	Unit 3
get method: A method that returns the value of an attribute is known as a get method.	Unit 10
Grid: The set $\{1, 2, 3, 4\} \times \{1, 2, 3, 4\}$, used in examples in Unit 7.	Unit 7
group of conditional statements, complete: By complete, we mean that at least one statement in the group will always be executed.	Unit 5
group of conditional statements, exclusive: By exclusive, we mean that only one if condition can be true (whatever the states of the variables involved in the conditions).	Unit 9
hash code: An integer associated with a string, used in storage.	Unit 6
hash table: A form of collection based on the hash code of a string: in M263, a hash table is an array of vectors of strings — each vector stores strings with the same hash code.	Unit 6
header: A specification preceding an implementation may be referred to as a header for the code.	Unit 2
heap sort: Starting with a sequence, add its items one at a time to a heap. Then form a new sequence by repeating the following actions: remove the root item from the heap and place it at the end of the vector, then reform the remaining items into a new heap.	Unit 8
heap tree: A complete tree that is sorted so that smaller values appear higher in the tree. The value at every child node must be greater than (or equal to) the value stored at its parent.	Unit 8
Idempotent laws: $s \vee s \equiv s$; $s \wedge s \equiv s$.	Unit 10
identifier: Each variable used in code has an identifier that the computer can recognise as a reference to that variable. Similarly, each process that may be called in code has an identifier the computer can recognise as a reference to that process.	Unit 1
imperative: A programming language that shares general features of the course code such as assignment, loops and conditionals is known as imperative.	Unit 2

implementation (of a class): A class is an implemented data type. The implementation of a class must be in terms of available types (i.e. types that are primitive or have already been implemented). The implementation of a class must include implementations of: the attributes of the class; the default state of the class and each method of the class.	Unit 4
implementation (of a function): A specification gives an overview of the effect of a function. Code to achieve this effect is called an implementation of the specification.	Unit 2
implementation (of a method): A specification gives an overview of the effect of a method. Code to achieve this effect is called an implementation of the specification. In implementing a method, one can access the attributes of the class, and can set the state of an object by setting the value of each attribute.	Unit 4
implementation and extension: If A extends B , then the implementation of any method of B is inherited by A . So is the implementation of each attribute of B , and the default values assigned to each attribute. (These features may be overridden, by additional code in the implementation of B . Usually, one will use the same implementation of an inherited method. But if one does change the implementation, then, in M263 code, one must make sure that the effect on the attributes inherited from class B is not changed.)	Unit 4
implication rule: This is the rule $\frac{s \Rightarrow t; s}{t}$.	Unit 10
implicit functional dependency: Implicit functional dependencies include trivial dependencies such as $\{x\} \rightsquigarrow \{x\}$ (which would hold in any scheme with an attribute x), and any dependencies that can be deduced from the explicit and trivial dependencies using rules like transitivity.	Unit 11
implies: In logic, a proposition $p \Rightarrow q$ can be read as “ p implies q ”, or as “if p then q ”.	Unit 9
index: In a sequence, items are indexed, with the first item (shown on the left) having index 1. So, for example, in $[5, 12, 14, 6, 7, 8]$, the item 7 has index 5. This indexing is used in the type functions of Array and in the methods of Vector .	Units 3, 6
induction hypothesis: In a proof by induction, we need to show that $p(k - 1) \Rightarrow p(k)$, and to do this, we typically “assume” that $p(k - 1)$ is true. In this context, we call $p(k - 1)$ the inductive hypothesis.	Unit 14
inductive hypothesis: An alternative term for induction hypothesis.	Unit 14
inductive step: In a proof by induction, the inductive step is showing that $p(k - 1) \Rightarrow p(k)$ is true .	Unit 14
inference (rules of): Rules of inference identify some common patterns in valid arguments (see page 15 of this <i>Handbook</i>).	Unit 10
infix: Included in the signature line of a function specification when the function is called using infix notation.	Unit 2
infix notation: In infix notation, the symbol that represents the process appears between the two items being combined. For example, we write the sum of the integers 2 and 7 as $2 + 7$. We also use infix notation when writing comparisons, such as ‘ a ’ < ‘ c ’.	Unit 1

inheritance: The course uses the term “inherits” for features (such as attributes and methods) that come to class A from class B when A extends B . In general, inheritance is a much broader concept than this, but the course does not discuss these more general ideas.	Unit 4
initial condition: An initial condition of a recurrence system gives the value of the function for a particular input, and serves as a stopping condition in a recursive evaluation of the function.	Unit 13
initialisation: Setting a starting value for a variable in code (where the type has no default value).	Unit 2
in-order tree traversal: A sequence giving the items at the nodes in the tree. To form this, write down: the sequence corresponding to the left subtree, then the root, then the sequence corresponding to the right subtree. (An empty tree gives an empty sequence.) Alternatively, use pancaking, or a tree walk.	Units 8, 13
input set: The set giving the form of data of the inputs to a function.	Unit 1
instance of a class: An object (variable) whose type is that class.	Unit 4
<i>Int</i> : The set of integers.	Unit 1
integer: A whole number (positive, negative or zero).	Unit 1
integer division: In integer division of n by d , the returned value, written $DIV(n, d)$, is the largest integer such that $d \times DIV(n, d)$ does not exceed n .	Unit 1
interior node (of a tree): A node that is neither the root nor a leaf.	Unit 8
invalid code: Any code that does not follow the rules of the course code.	Unit 2
invalid expression: An expression that uses a function in a way that is not consistent with its signature or its precondition is invalid.	Unit 1
inverse functions: Functions F and G are inverse functions if F reverses the effect of G , and G reverses the effect of F (as do, for example, <i>ASC</i> and <i>CHR</i>).	Unit 1
key of a relation scheme: In a relation scheme, a subset K of the set of attributes Att is a key for the relation scheme if both of the following properties hold. For every attribute x in Att , $K \rightsquigarrow \{x\}$. (Determining) There is no proper subset $Y \subset K$ with the determining property. (Minimal)	Unit 11
leaf node: A node with no branch leading out below it.	Unit 8
left child (of a node in a binary tree): The child at the end of the left branch below the node.	Unit 8
left subtree: In a non-empty binary tree, there are two subtrees (these may be empty). They are distinguished as the left subtree and the right subtree, and the left subtree is shown in diagrams at the end of the branch below and to the left of the root.	Unit 8

length (of a sequence): The number of members in the sequence.	Unit 1
level (in a tree): For any node in a tree, the number of branches one must follow to return to the root determines its level. (The root itself is at level 1, the children of the root are at level 2, and so on.)	Unit 8
level order tree traversal: Given a tree, we form a sequence containing the items at its nodes as follows. (The sequence is written down going from left to right.) First, the root. Then the nodes at level 2, working from left to right. Then the nodes at level 3, working from left to right. This process is continued, going down one level at a time, until all the nodes in the tree have been visited.	Unit 8
lexicographic order: “Dictionary” order for strings.	Units 6, 8
library class: The WorkPad makes available a number of classes that are already implemented. We refer to these as library classes.	Unit 4
linear notation (for a tree): In M263, a way of expressing a tree that does not require a diagram.	Unit 8
linear polynomial: A polynomial (in n) is linear if its term with the highest power of n is a constant multiple of n .	Unit 15
linear search: A search examining items in a sequence in sequential order.	Unit 15
literal: A constant value written in a notation that the computer can interpret.	Unit 2
local variable: A variable declared within an implementation is said to be local to that implementation.	Unit 2
loop: A code statement in which one or more statements are to be repeated.	Unit 2
loop invariant: A quantity (involving variables) that is not changed by a single execution of the body of a loop.	Unit 14
loop invariant condition: A condition that holds after completion of each execution of the body of a loop.	Unit 14
loop variable: In a loop starting for ($index \leftarrow m$ to n), $index$ is called the loop variable.	Unit 2
many to many association: Two relation schemes are said to have a many to many association if a tuple in either scheme can correspond to several tuples in the other scheme.	Unit 11
marker (of a turtle): The feature of an object of type Turtle giving its position and direction it is facing. In the implementation, the corresponding attribute is of type Marker .	Unit 5
marker state invariant: A code fragment (involving a single variable of type Turtle) for which execution of the code fragment leaves the marker state unchanged is called marker state invariant.	Unit 5
mathematical induction: A proof technique involving a starting step and an inductive step. The technique can be used to prove results about integers, or results that can be related to integers (for example, by considering the length of a sequence or the depth of a tree.	Unit 14

member of a sequence: The items in a sequence may be referred to as the members of the sequence.	Unit 1
member of a set: The items in a set may be referred to as the members of the set.	Unit 1
method: A type function of a class. A method is part of the specification of a particular class, and can only be applied to an object of the class of which the method is part. A statement in which a method (say <i>aMethod</i>) is applied to an object (say <i>anObject</i>) is shown by the dot notation, as in <i>anObject.aMethod()</i>	
A method may require one or more parameters. The type of any parameters will be given as part of the method specification. Parameter values must be given in brackets in a statement calling that method, so that the general form is <i>anObject.aMethod(parameters)</i> .	
In M263, a method will either return a value or will change the state of the object to which it is applied. (M263 deals with a restricted concept of “method”. Most programming languages permit methods both to return a value and to change the state of an object.)	Unit 4
minimal property: See entry for “key of a relation scheme”.	Unit 11
modulo: The remainder $MOD(n, d)$ on integer division of n by d may be referred to as the value of n modulo d .	Unit 1
modus ponens: Another name for the implication rule.	Unit 10
modus tollens: Another name for the contrapositive rule.	Unit 10
negation: The function <i>NOT</i> on <i>Bool</i> or $\neg$ applied to a proposition.	Units 1, 9
nested conditional statement: If the bracketed statements after if or else themselves involve further if statements, we refer to the entire conditional statement as nested.	Unit 9
node (of a tree): A location in a tree.	Unit 8
normal form, first: We say that a table is in first normal form if it has no duplicate rows and a single-valued entry in each cell.	Unit 11
normal form, second: We say that a relation scheme is in second normal form if it is in first normal form and we cannot find a key K , an attribute x that is not in K , and a proper subset P of K , with $P \rightsquigarrow \{x\}$.	Unit 11
normal form, third: We say that a relation is in third normal form if it is in second normal form and has no transitive dependencies.	Unit 11
n-tuple: An association of n items, each taken from a specified set. An n -tuple comes from a Cartesian product set. For example, the 3-tuple (<i>‘a’</i> , 5, “tring”) is from $Char \times Int \times SeqOfChar$.	Unit 1
object: A variable of class type (also known as an instance of the class).	Unit 4
one to many association: Two relation schemes are said to have a one to many association if a tuple in the first scheme can correspond to several tuples in the second scheme.	Unit 11

one to one association: Two relation schemes are said to have a one to one association if each tuple in the first scheme can correspond to only one tuple in the second scheme.	Unit 11
order (of a function): In M263, the order of a function $T(n)$ is a function, say $f(n)$, whose behaviour for large values of n is equivalent to that of $T(n)$ (in that $T(n)$ asymptotically dominates $f(n)$, and $f(n)$ asymptotically dominates $T(n)$). We say that $T(n)$ is $\Theta(f(n))$. The order will be taken from a standard hierarchy (see page 21 of this <i>Handbook</i>).	Unit 15
ordered pair: An association of two items, each taken from a specified set (also known as a 2-tuple). An ordered pair comes from a Cartesian product of two sets.	Unit 1
ordered triple: An association of three items, each taken from a specified set. An ordered triple comes from a Cartesian product of three sets.	Unit 1
outcome table: An outcome table tabulates the outcomes of a piece of code for all possible values of the Boolean variables within the code.	Unit 9
outcome tree: An outcome tree shows the outcomes of a piece of code for all possible values of the Boolean variables within the code in the form of a tree. It is usual to draw the tree on its side.	Unit 9
output: The value returned by a function may also be called the output of the function.	Unit 1
output set: The set giving the form of data of the value returned by a function. For example, if the function returns an integer, then the output set is <i>Int</i> .	Unit 1
overloading: The use of the same identifier for different processes, such as the use of <code>==</code> for equality tests both on integers and on characters.	Unit 3
pancaking a tree: If a binary tree is drawn in a way that ensures that we can draw a line vertically down from each node without crossing the tree, then “squashing” the tree vertically will give the in-order traversal. This is called “pancaking” the tree.	Unit 8
parameter: A method may require one or more parameters. When a method is called, the parameter values are given in brackets after the method identifier, as <i>aMethod(parameters)</i> .	Unit 4
parameterised type: A data type such as Stack of X that has another type (X) as a parameter.	Unit 3
parent node: If two nodes in a tree are connected by a branch, the node nearer to the root is called the parent, and the node further from the root is called the child.	Unit 8
parse table: A parse table is used to show the process of building up a well-formed formula. In the table, we give a reference number and, when the formula is not an atom, the rule that we have used for each well-formed formula in the list.	Unit 9

parse tree: In a parse tree, each leaf node is an atom. Each non-leaf node is an operation, and the branches from such nodes lead either to an atom or to a subtree that is again a valid parse tree. If the operation is $\vee$ or $\wedge$, the node has two branches, while if it is $\neg$, the node has one branch.	Unit 9
pen (of a turtle): The feature of an object of type Turtle that determines whether or not spots are added to the diagram when <i>forward</i> is called. In the implementation, the corresponding attribute is of type Bool .	Unit 5
platform independent: A measure of efficiency that can compare programs in a way that does not depend on the computer they are run on is said to be platform independent.	Unit 15
polynomial: A polynomial (in n) is an expression of the general form $a_1n^c + a_2n^{c-1} + \cdots + a_{c-1}n + a_c$, where c is a positive integer.	Unit 15
postcondition (of a function): The postcondition is given in the final line of a function specification. It is the responsibility of a user to make sure that the function is called with the correct number of inputs, of the correct type, and in the appropriate order (as given in the specification), and that these inputs satisfy the precondition. If that is done, then the output will satisfy the postcondition.	Unit 2
postcondition (of a method): The postcondition is given in the final line of a method specification. It is the responsibility of a user to make sure that the method is called in a way consistent with the signature and precondition. If that is done, then the effect of calling the method will be consistent with the postcondition. (The effect of the method may be either to return a value, or to change the state of the receiver object.)	Unit 4
post order tree traversal: A sequence giving the items at the nodes in the tree. To form this, write down: the sequence corresponding to the left subtree, then the sequence corresponding to the right subtree, then the root. (An empty tree gives an empty sequence.)	Unit 13
power set: The set <i>SetOfX</i> , consisting of all sets whose members are from X , is called the power set of X .	Unit 1
precedence of negation: This is the rule of precedence that says that terms of the form $\neg r$, where r is a Boolean expression, should be calculated as soon as the value of r has been found. M263 uses this rule of precedence.	Unit 9
precondition (of a function): A user must call a function with an appropriate input. To recognise an appropriate input, look at two things. The first line of the specification shows how many inputs the function requires, the order in which they must be given, and what form of data they must have. The second line of the specification shows the precondition, which gives any additional conditions that the input(s) must satisfy.	Unit 2
precondition (of a method): The precondition of a method shows any additional conditions required on the state of the receiver object or on parameter value(s).	Unit 4

predefined set: In set comprehension, we define a set by use of a predicate, as in $\{x \text{ in } X : p(x)\}$. In this context, X is called a predefined set — X must have already been defined.	Unit 7
predicate: The word predicate is used to refer to a property of a single item or to a relationship between several items (see page 14 of this <i>Handbook</i>).	Units 7, 10
predicate sentence: A predicate sentence involves a predicate and variable(s) (e.g. $x < 5$, where x is in <i>Int</i>). Usually, we do not take care to distinguish between a predicate and a predicate sentence.	Unit 7
premises: In an argument corresponding to an expression $(s_1 \wedge s_2 \wedge \dots \wedge s_n) \Rightarrow t,$ we call $s_1, s_2, \dots, s_n$ the premises of the argument.	Unit 10
pre-order tree traversal: A sequence giving the items at the nodes in the tree. To form this, write down: the root, then the sequence corresponding to the left subtree, then the sequence corresponding to the right subtree. (An empty tree gives an empty sequence.)	Unit 13
primary key: Where a relation scheme has more than one key, it is usual to declare one of these as the primary key of the relation scheme.	Unit 11
prime: An integer $p (> 1)$ is prime if it has no factors (apart from 1 and p itself).	Unit 3
primitive data type: Data types (such as Int , Char and Bool) that are not class types, and are assumed to be available by the course code (and for which no implementations are described).	Unit 3
primitive form of data: We refer to unstructured forms of data, such as integers, characters and Booleans, as primitive forms of data.	Unit 1
priority queue: A queue in which every item has an associated priority. A heap can be used to model a priority queue.	Unit 8
process: A general term used in the course to refer to manipulation of data.	Unit 1
proof: By a proof of an argument we mean a finite sequence of propositions, each of which is either a premise or can be deduced from the previous propositions, ending with the conclusion.	Unit 10
proof of correctness: A proof that an implementation is correct.	Unit 14
proper subset: A is a proper subset of B , written $A \subset B$, if A is a subset of B , and A is not equal to B .	Unit 7
proposition: An assertion (about conditions in the outside world) that may be either true or false is called a proposition.	Unit 9
quadratic polynomial: A polynomial (in n) is quadratic if its term with the highest power of n is a constant multiple of n^2 .	Unit 15
quantifiers: These are the universal and existential quantifiers that are used to enrich the logical language that we use.	Unit 10
quotient: The result of integer division is called the quotient.	Unit 1

random access: A collection type in which any member of the collection can be easily accessed, e.g. Array or Vector , is said to have random access.	Unit 6
real number: The wider collection of numbers, including for example $122/7$, $\sqrt{2}$ and $\pi(= 3.14159\dots)$, form the real numbers. A real number is conveniently thought of as a decimal.	Unit 3
receiver: The object to which a method is addressed (in a statement using the dot notation).	Unit 4
recurrence system: A recurrence system provides a recursive description of a function with signature Int $\rightarrow$ Int , and a suitable precondition.	Unit 13
recursion: The use of self-reference in either an implementation of a function, or the formal expression of its postcondition.	Unit 13
recursive call: In a recursive algorithm, a statement in which the function being implemented is called for a different (typically smaller) input. In any recursive description, the part where the function calls itself with a different input.	Unit 13
recursive part (of a recurrence system): The self-referential part of a recurrence system.	Unit 14
reference: To be precise, one should say that a variable identifier references an object (because the identifier holds the memory location of the object rather than the object itself), but in M263 we do not consider this feature of data storage.	Unit 4
references, foreign key: See foreign key.	Unit 11
relation: A collection of tuples forming a complete table is a relation.	Unit 11
relation scheme: We use the term relation scheme to mean a set of attributes <i>together with</i> any underlying rules that apply. A relation over the relation scheme must obey these underlying rules.	Unit 11
relational database: A relational database consists of a collection of relation schemes linked together.	Unit 11
relative complement: When B is a subset of A , the set difference $A - B$ is called the relative complement of B in A .	Unit 7
remainder: In integer division, the remainder when n is divided by d (written $MOD(n, d)$) is $n - d \times DIV(n, d)$, and is a quantity less than d , but not less than 0.	Unit 1
representation (of a data abstraction): A form of data derived from an application situation may be referred to as a data abstraction, in distinction to its representation in terms of forms of data available on a computer.	Unit 1
reserved word: A word used in the course code with a special meaning (such as var or if). Reserved words must not be used as variable identifiers.	Unit 2
returned value: A function takes values of specified types as inputs, and produces a corresponding value, known as the output or returned value.	Unit 1

right child (of a binary tree): The child at the end of the right branch below the node.	Unit 8
right subtree: In a non-empty binary tree, there are two subtrees (these may be empty). They are distinguished as the left subtree and the right subtree, and the right subtree is shown in diagrams at the end of the branch below and to the right of the root.	Unit 8
root (of a tree): The node shown at the top of a tree diagram.	Unit 8
Rules about implication: $s \Rightarrow t \equiv \neg s \vee t$; $s \Leftrightarrow t \equiv (s \Rightarrow t) \wedge (t \Rightarrow s)$.	Unit 10
Rules about negation: $s \vee \neg s \equiv \text{T}$; $s \wedge \neg s \equiv \text{F}$; $\neg \neg s \equiv s$.	Unit 10
scope: In a quantified expression, the part in square brackets is known as the scope of the quantifier.	Unit 10
selector predicate: The predicate p used in the condition when a set is defined by comprehension, as $\{x \text{ in } U : p(x)\}$.	Unit 7
semantic consequence: In an argument corresponding to an expression of the form $(s_1 \wedge s_2 \wedge \dots \wedge s_n) \Rightarrow t$, if the expression is a tautology, so that we have a valid argument, then t is called a semantic consequence of $s_1 \wedge s_2 \wedge \dots \wedge s_n$.	Unit 10
semantics (of a function): A description of the effect of a function (in terms of the relationship between the input and returned values), usually given in a postcondition.	Unit 1
sequence: A collection of items, in a particular order, and in which repetitions are of significance.	Unit 1
sequential search: An alternative term for linear search.	Unit 15
set: A collection of items in which repetition is not significant, nor is the order in which the items are given.	Unit 1
set comprehension: The definition of a set (say A) by giving a pre-defined set and a condition that members of A must satisfy: for example, $\{n \text{ in } \text{Int} : n > 2\}$ is the set of integer greater than 2.	Unit 7
set difference: The set difference $A - B$ consists of those elements in the set A that are not in the set B .	Unit 7
set enumeration: A set is given by enumeration if we write down each of its members (e.g. $\{1, 4, 7, 9, 12\}$).	Unit 7
set equality: Two sets are equal if they contain exactly the same members.	Unit 7
set intersection: The set intersection $A \cap B$ consists of those elements that are in both the set A and the set B .	Unit 7
set method: A method that changes the state of an object by providing values for one or more of the attributes is known as a set method.	Unit 5
set union: The set union $A \cup B$ consists of the elements in either or both of the sets A and B .	Unit 7

side effect: If calling a function in user code has the effect of changing the state of the input variables, then we say that the function has a side effect. In M263, functions should not have side effects. It is not valid to specify that a function should have a side effect, and any implementation of a function with a side effect is not correct.	Unit 4
sigma notation (for a sum): A notation that expresses a summation using the Greek letter capital sigma (Σ).	Unit 14
signature: The signature of a function gives the input and output sets of the function, in the form (for example)	
$FIRSTAT : Char \times SeqOfChar \rightarrow Int.$	Unit 1
signature line: The first line in a specification, from which the signature can be deduced. For a function, the signature line gives the types of the inputs and the type of the returned value (and states whether infix notation is used). For a method, the signature line gives the type of any parameters, and states whether it returns a value or changes the state of the receiver.	Units 1, 4
simple proposition: An assertion (about conditions in the outside world) that may be either true or false and does not use any connectives is called a simple proposition.	Unit 9
simplification rule: This rule says that if we know that both s and t are true then we know that s is true .	Unit 10
solution (of a recurrence system): A solution of a recurrence system is a formula giving the same values as the function defined by the recurrence system.	Units 13, 14, 15
sorted sequence: A sequence of integers is sorted if the integers appear in the sequence in increasing order. In general, a sequence is sorted if $a \leq b$ holds whenever a comes immediately before b in the sequence. The interpretation of $\leq$ here can come from any relationship for ordering items of a particular type — for example, strings are compared using the lexicographic order.	Unit 6
sorts: In the formal axiomatic specification of an ADT, the sorts are the types and sets used.	
source set: An alternative name for the input set of a function.	Unit 1
specification (of a function): A “black box” view, describing the behaviour of a function. Function specifications in M263 are given in the form: signature line, precondition, postcondition.	Unit 2
specification (of a method): A description of the behaviour of a method. In M263, methods either change the state of the receiver object, or return a value; they never do both, and the specification is given in the form: signature line, precondition, postcondition.	Unit 4
stack: A variable of type Stack of X is referred to as a stack. The form of data of a stack is a sequence whose size is not fixed. All the items in a stack will be of the same type. A stack variable has access to the type operations <i>SIZE</i> , <i>PUSH</i> , <i>POP</i> and <i>PEEK</i> .	Unit 3

starting condition: Another term for the starting step or basis for induction in a proof by mathematical induction. Also used as an alternative term for the initial condition of a recurrence system.	Unit 14
starting step: In a proof by mathematical induction, establishing the result for a particular value — establishing that $p(M)$ holds for a particular integer M , as a basis for showing that $p(n)$ is true for all integers $n \geq M$.	Unit 14
state: The data stored in a variable (of either primitive type or class type) at a given moment.	Units 1, 4
statement: A piece of code that may be addressed to a computer, describing a calculation that is to be performed.	Unit 2
statement of requirements: This is a list, usually written in natural language, of properties that a piece of software must possess.	Unit 9
stopping case: In a recursive description, a part where the function value is given for a particular input (in a way that enables the recursion to terminate). There may be more than one stopping case.	Unit 13
stopping condition: In a recursive description, a condition on the input value that leads to a stopping case.	Unit 13
stopping value: In a recursive description, the value returned by the function in a stopping case.	Unit 13
string: A sequence of characters.	Unit 1
strongly typed: Features of a strongly typed programming language include: every variable has a type; every process has a signature; process application must be consistent with the signature of the process; assignment must be consistent with type. (Processes may share an identifier — see overloading.)	Unit 3
subexpression: A part of an expression that is in itself representable as a parse tree.	Unit 9
subset: If every member of the set A is also a member of the set B , then A is a subset of B , written $A \subseteq B$. (If A and B are equal sets, we still have $A \subseteq B$.)	Unit 7
substring: A substring of a string is a continuous run of characters taken from the string.	Unit 6
subtree: A part of a tree that is itself a tree.	Unit 8
syllogism: An argument with two premises and one conclusion is known as a syllogism.	Unit 10
syntax: The rules of how valid expressions are constructed.	Unit 3
syntax (in formal specification): In the formal axiomatic specification of an ADT, the syntax comprises the statements of the signatures of the methods (or type functions).	Unit 16
syntax (in predicate calculus): Expressions in predicate calculus have a syntax, just as do Boolean expressions. Essentially, we need the expression inside the square brackets following a quantifier to be syntactically correct as a Boolean expression.	Unit 10

target set: An alternative name for the output set of a function.	Unit 1
tautology: An expression that is always true , no matter what the truth value of its constituent variables, is called a tautology.	Unit 10
termination: To be valid, a recursive description must ensure termination, i.e. that for all permitted inputs, a stopping condition will be reached after a finite number of recursive calls.	Unit 13
testing (a function implementation): Tracing (or executing) an implementation for one or more particular inputs (that satisfy the precondition), and checking that the returned value satisfies the postcondition.	Unit 2
this: In expressing the implementation of a method of a class, the receiver object is given the identifier <i>this</i> .	Unit 4
time complexity function: In M263, the time complexity function $T(n)$ is a measure associated with an implementation expressed in the course code, and is a count of the total number of statement executions for an input of size n , in the worst case.	Unit 15
time efficiency function: An alternative term for the time complexity function.	Unit 15
total function: A function with precondition true , i.e. a function that is defined for every member of the input set.	Unit 1
tracing code: The action of following the values taken by the variables in code. This may be done statement-by-statement, but in code containing a loop it is more typical to give the states of the variables once for each execution of the loop body.	Unit 2
transitively dependent: An attribute a is transitively dependent on a set of attributes X , where X is a key of the scheme, if there is another set of attributes Y such that $X \rightsquigarrow Y \text{ and } Y \rightsquigarrow \{a\} \quad \text{but} \quad Y \not\rightsquigarrow X \text{ and } a \notin X \cup Y.$	
Such a pair of dependencies form a transitive dependency.	Unit 11
tree: A tree is a collection of nodes joined by branches. It has one node called the root, and every other node is connected to the root by a unique path through the branches.	Unit 8
tree traversal: A systematic way of visiting each node in a tree once.	Unit 8
trivial functional dependency: These are dependencies such as $\{x\} \rightsquigarrow \{x\}$ (which would hold in any scheme with an attribute x).	Unit 11
truth table: A table that gives the truth value of a Boolean expression for every possible combination of truth values of the variables.	Unit 9
tuple: A row in a table of data is called a tuple. Each tuple gives a data item — an entry in the database.	Unit 11
turtle: An object of type Turtle .	Unit 5
TurtleBox: A class used in a case study in Unit 10.	Unit 10
type: Each variable in code has a type. This is a data type (either a primitive type or class type), and is given in the var statement declaring the variable.	Unit 3

type consistent: An assignment statement is said to be type consistent if the data being stored is of the same type as the variable. An expression is type consistent if any function calls are consistent with the signature of the function.	Unit 2
type function: The functions associated with a data type are called its type functions.	Unit 3
UNICODE: The current international standard coding for characters, of which the ASCII codes used in this course are just a part.	Unit 1
universal quantifier: The universal quantifier, which is denoted by $\forall$, can be read as “for all”. The variable following the quantifier must come from a set within the domain of interest.	Unit 10
universe of discourse: An alternative term for the domain of interest.	Unit 10
valid argument: An argument with premises $s_1, s_2, \dots, s_n$ and conclusion t is a valid argument if $(s_1 \wedge s_2 \wedge \dots \wedge s_n) \Rightarrow t$ is a tautology.	Unit 10
valid recursion: Requirements for valid recursion are: one or more stopping conditions; one or more recursive calls; and termination.	Unit 13
variable: A named area of memory storing data that may change.	Unit 1
variable, bound: A variable that is restricted by a quantifier is known as a bound variable.	Unit 10
variable, free: If a Boolean expression contains a variable and that variable is not restricted by a quantifier, we call the variable a free variable.	Unit 10
vector: A sequence value giving the state of an object of type Vector . In the vector $[1, 2, 3, 4, 5]$, we call 1 the first element (at the front of the vector) and 5 the last element (at the back of the vector).	Unit 6
Venn diagram: A form of diagram used to illustrate sets.	Unit 7
well-formed formula: A well-formed formula is a Boolean expression that is syntactically correct.	Unit 9
while condition: The condition following the keyword while that controls execution of a while loop.	Unit 6
WorkPad: The software package supplied with this course, which expresses the course code in a way that can be executed on a computer.	Unit 3
WorkPad class library: The classes available in the WorkPad.	Unit 5
worst case (for time complexity): In determining the time complexity $T(n)$, the worst case relates to an input of size n for which the total number of statement executions is greatest.	Unit 15

11 DATA TYPE SUMMARIES

Primitive types — type functions

Int	$x + y; x \times y; x - y; x == y; x < y; x > y; x \leq y; x \geq y; DIV(x, y); MOD(x, y)$
Bool	$NOT(x); x \wedge y; x \vee y; x == y$
Char	$ASC(c); CHR(n); c == d; c < d; c > d; c \leq d; c \geq d$
Array of X	$SIZE(arr); AT(i, arr); PUT(i, arr, item)$
Stack of X	$SIZE(s); PUSH(x, s); POP(s); PEEK(s)$

Class types — methods

Spot	$getXPos(), getYPos(), upOne(), downOne(), leftOne(), rightOne()$
QuickSpot	From Spot plus $upQuick(n), downQuick(n), leftQuick(n), rightQuick(n)$
ColourSpot	From Spot plus $getColour(), nextColour()$
Marker	From Spot plus $nextDirection()$
Turtle	$forward(n), leftTurn(), rightTurn(), penOn(), penOff(), store(), restore()$
TurtlePlus	From Turtle plus $rectangle(side1, side2)$
SuperTurtle	From TurtlePlus plus $diagonal(n)$
Vector of X	$setVector(list), size(), addFirst(x), addLast(x), setAt(n, x), removeFirst(), removeLast(), isEmpty(), contains(x), first(), last(), at(n)$
VectorPlus	From Vector plus $remove(n), insert(n, x), sort(), join(v)$
mString	$setString(listOfChar), length(), charAt(i), substring(n, m), substr(n, len), charAppend(c), concat(s), equal(s), hashCode()$
Set of X	$setSet(list), addElement(x), card(), contains(x), containsSet(A), equals(A), union(A), intersect(A), minus(A), cartProd(B)$
Pair of X,Y	$setPair(x, y)$
BinTree of X	$setTree(x, l, r), setRoot(x), setLeftTree(l), setRightTree(r), getRoot(), getDepth(), isEmpty(), leftTree(), rightTree()$

12 ASCII CODES FOR CHARACTERS

Code		Code		Code		Code		Code		Code	
32		48	0	64	@	80	P	96	'	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	\$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	-	111	o		